



MASTER THESIS

Fault-Tolerant Microarchitectures: Enhancing Reliability through Redundant Execution

Author : DEVROYE Louis

Advisor : MÜHLBERG Jan Tobias

Director : MÜHLBERG Jan Tobias

Academic year 2025–2026

Abstract

TODO 1: ABSTRACT AT THE END

Contents

1	Introduction	4
2	State of the Art	6
2.1	Engagment in the scientific community	6
2.2	Faults	6
2.3	Fault Recovery Techniques	7
2.3.1	Space Redundancy	7
2.3.2	Time Redundancy	8
2.3.3	Fault Detection	9
2.3.4	Fault Injection	9
2.4	System's criticality	10
2.5	Cybersecurity	10
2.5.1	Trusted Execution Environments (TEE)	10
2.5.2	Enclave	10
2.5.3	Code obfuscation & diversification	11
2.5.4	Rewinding & Domains	11
2.5.5	Data & Control flow	11
2.5.6	Obfuscation measurement	11
2.5.7	Limitations	12
2.5.8	Background work	12
3	Problem Statement and Objectives	13
4	Keystone Structure	15
4.1	RISC-V	15
4.1.1	Physical Memory Protection (PMP)	16
4.1.2	Security Monitor (SM)	16
4.1.3	Interrupts and Exceptions	16
4.2	Basics	17
4.2.1	Overview	17
4.2.2	Workflow	18
4.3	Enclaves	19
4.3.1	Enclave lifecycle	19
4.4	Quick EMUlator (QEMU)	20
4.4.1	Eyrie runtime	20

4.4.2	FireSim	20
4.5	Edge calls	20
4.5.1	OCALL lifecycle	20
4.6	Data sealing	23
4.6.1	Key-Hierarchy	24
4.6.2	Key Derivation	24
4.7	Compilation	25
4.7.1	Rebuilding Changed Components	25
4.7.2	Platform	26
4.7.3	Bits	26
4.7.4	Builroot Config file	26
5	System Model and Design	27
5.1	Preliminary setup and selection	27
5.2	Threat Model	28
5.2.1	Trusted	28
5.2.2	Adversarial	28
5.2.3	Attacker Capabilities	28
5.2.4	Security Objectives	28
5.2.5	Accepted Limitations	28
5.2.6	Assumptions	28
5.2.7	Requirements	29
5.3	Implementation	29
5.3.1	Checkpoint	32
5.3.2	Data encryption	37
5.3.3	Fault Modeling	40
5.3.4	Scripts	41
6	Experimental Setup	42
6.1	Macros	42
6.1.1	Logging	43
6.1.2	Execution	43
6.1.3	Testing	43
6.2	Test cases	44
6.2.1	Cyber security	44
6.2.2	Analysis	44
7	Results and Discussion	45
7.1	Static Analysis	45
7.1.1	Computation threshold	46
7.2	Runtime Analysis	48
7.2.1	Computation threshold	48
7.2.2	Cycle cost per operation	51
7.3	Cyber security	52

7.3.1	Security objectives	52
7.4	Discussion	53
7.5	Future Work	54
7.5.1	Mixed criticality	54
7.5.2	Modern lightweight cryptoscheme	54
7.5.3	More complex fault model	54
7.5.4	Enclave Complexity on backup	54
8	Conclusion	55

Chapter 1

Introduction

Microprocessors are getting more efficient each year due to miniaturization of transistors [1]. However, this rise in efficiency comes with the pricey cost that are faults. Critical systems are particularly vulnerable considering the stakes (human lives) and the real time constraints such as time, temperature, computational power or even voltage. This field is not new [2, 3] but is still drawing attention, has a lot of gaps yet to be covered and is structuring, mostly around the type of faults and the way to deal with them [4].

Fault detection is a widely studied field [5], it is a pillar for critical systems and for networks. The detection goes along with Error Correction Codes (ECC), these are mostly used in network due to the necessity to get the good informations using as little of time/computation as possible. However, these techniques come with a lot of overhead and assumptions that cannot be made in this field. First, network consider the source to be viable, when a request is made to a server, the informations are checked, let's say with a basic checksum. In micro architectures, the sensors would be the parallel to the server. However, the data given by the sensor are not known to be true or false (exception made for specific cases, for example if a distance check returns a negative answer). Secondly, all the layers needed to prevent a fault from happening, in TCP for example, would be too costly to just put in a micro processor, in overhead (from both computation and memory standpoint) and in time needed. In fact, the main topic is that these systems are time bounded, so it is important to be able to have the, correct, data before a certain deadline. Unfortunately, all these constraints do not apply, at least not all at once, to networks.

On the other hand, a lot of programs and studies are focused on *Software Implemented Fault Tolerance* (SIFT) and more precisely on *time redundancy* methods, they are solely interested in making fault-tolerant programs that meet deadlines. That is in itself an honorable (and complex) goal, yet one of the main problem, related to this topic, with miniaturization being the increase in temperature. One could argue that making extra computation or overclock the CPU when an error occurs, possibly because of the aforementioned heat, wouldn't make any sense. In fact, it could slow the whole system (to cool down) or cause more fault at the time being and it would strain the system even more on the long run [6, 7].

To improve this contradictory aspect, some research has been done. They put the light on other aspects than only timing constraints (while keeping the reliability). For example, power and

energy consumption as the *Internet of Things* (IoT) develops [8] is gaining more attention.

Hardware is the basis off all this domain and because of this, extensive work can be done directly on it to prevent or recover from faults directly on the hardware. This approach deletes most of the overhead of the software techniques but it is much more restricted and complex than implementing and debugging a layer over a "simple" hardware structure.

Moreover, this subject can be extended to *cybersecurity* in a way to harden the task (or at least the conception) of malicious programs [9]. In fact, an attacker could try to reverse engineer a program, he could also try to check the memory for data or the CPU usage and instructions or even using the power supply as an attack vector [10] to prevent a task from completing. For this purpose, several techniques have been developed to try and block either the static or dynamic analysis of a program [9]. Even measuring the redundancy of a program to be able to scale and compare two diversification technique and choose one for example [11]. The research could even separate the low-level redundancy from a high-level, algorithmic, one.

The codebase (implementation of the checkpoint enclave and tests) of this thesis can be found on the [GitHub repository](#) . This is a fork of the main Keystone github [Keystone repository](#) [12] . The files can also be found in the Appendix (8).

Chapter 2

State of the Art

2.1 Engagment in the scientific community

For as long as computers (some could even say tools in general) have existed, so have defaults, errors and bugs [2]. However, the need for a solid structure and concretes answers increased as Moore's law kept going on [13]. This topic is drawing more attention as the years go by.

2.2 Faults

Faults are the basis of the problem but the outcome a of fault, and they way it is handeled, is equally important. To be clear and rigorous, we will use the Institute of Electrical and Electronics Engineers' (IEEE) [4, 14, 6] standards. In this goal, we can separate a fault and its consequences in 3 categories each one following from the previous one:

- **Fault:** A defect in a hardware device or component. It can be a *Single Event Upset* (SEU), otherwise seen as a bit flip or it can be other models such as *Multiple Event Upset* (MEU).
- **Error:** The difference between a computed, observed or measured value or condition and the true, specified, or theoretically correct value or condition. It is the effect of a fault on the system.
- **Failure:** The inability of a system or component to perform its required functions within specified performance requirements. In other words, the program malfunction due to an error.

Now, we need to categorize faults [4, 6]

- **Transient:** Produces a temporary effect on the system in a very short period of time. It is often modeled using SEU. They are most likely cause by : -
 - alpha radiation (due to cosmic radiation or nuclear fission)
 - electromagnetic interference
- **Intermittent:** Lifespan similar to *Transient* but it occurs in *burst*. It is mainly caused by eternal conditions : temperature, voltage, hardware wear out and High Intensity Radiated-Field (HIRF)

- **Permanent:** Results from hardware wear out or manufacturing defects. The only fix for this type is to repair or change the hardware. Component wear out is well known in the current literature, the *Bathtub curve* [15] describe the three main phases of a component's life :
 - The *infant* stage is when the hardware is fairly new (not in term of design). Thus, it might be defective while still passing the basic tests required for using it. As this phase goes on, the average error rate decreases.
 - The *main* stage is, for the main part, the longest one. The component is viable (since it has passed the previous phase) and not yet worn out. But still random faults occurs (due to environment, etc.). This phase is the best in term of fault rate (which is supposedly constant).
 - The *wear out* stage is the first that comes to mind when speaking of permanent faults. A part of the component is defective and there is no direct fix except for replacing/repairing this part. In opposition to the 1st phase, the average error rate increases over time.

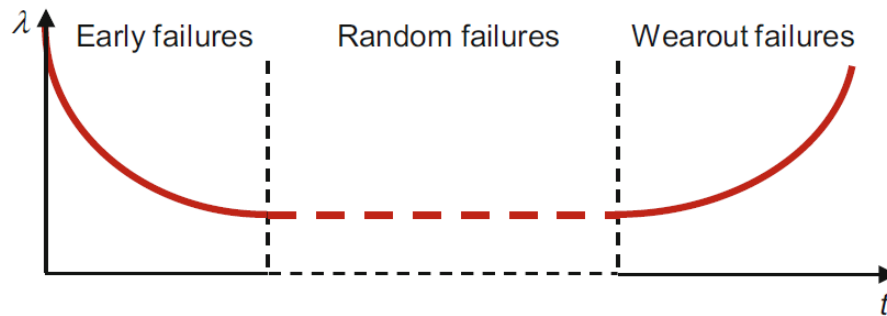


Figure 2.1: Bathtub curve. "Typical plot of the λ failure rate over time t (bathtub curve) with decreasing, constant, and increasing failure rate sections" - Fig4.4 (p52) [16]

2.3 Fault Recovery Techniques

2.3.1 Space Redundancy

- **N-modular redundancy (NMR):** this consists of N (2,3,4,..) instances of the job i running at the same time, when the job i finishes, a vote start and the majority is returned as the result. NMR can detect by essence and correct the error. However, it is important to note that it can only detect an *error*, not a *fault* [4]. Also, each n instance of the job i should run on different cores to avoid SEU.
- **standby redundancy:**
 - *Hot* standby is when the backup system runs in parallel with the main one. If something fails, the switch to the backup is almost instant since it is already active and synchronized.
 - *Cold* standby is when the backup system is off until the main one fails. When that happens, the backup starts up and takes over. This takes longer but saves resources while things are working.

2.3.2 Time Redundancy

- **re-execution:** This technique runs the same task again to see if the error happens in the same way. If the second run works fine, the first one probably had a temporary problem. It helps filter out transient faults. It has two main approaches:
 - *Backward error recovery:* If the state of the job or the environment needs to be restored after a fault, then the faulty job is restarted.
 - *Multiple recovery:* due to scheduling reasons, the currently preempted jobs are also re-executed even if not affected by the fault.
- **Checkpoint:** A checkpoint saves the state of the system at a certain time. If a failure happens later, the system can roll back to this saved state and continue from there without starting over from scratch. It is often used in High Performance Computing (HPC) for long running tasks and in the industry in general as it is effective.
- **Recovery blocks:** This method runs a primary version of a task and checks the result. If the result fails, it runs another version (possibly degraded) to try again. This can repeat with more versions if needed until one gives a good output.
- **Forward error recovery:** Mostly used in networks, it executes a special routine that fixes the error without re-executing the original task. Simple raise/throw exception handling.

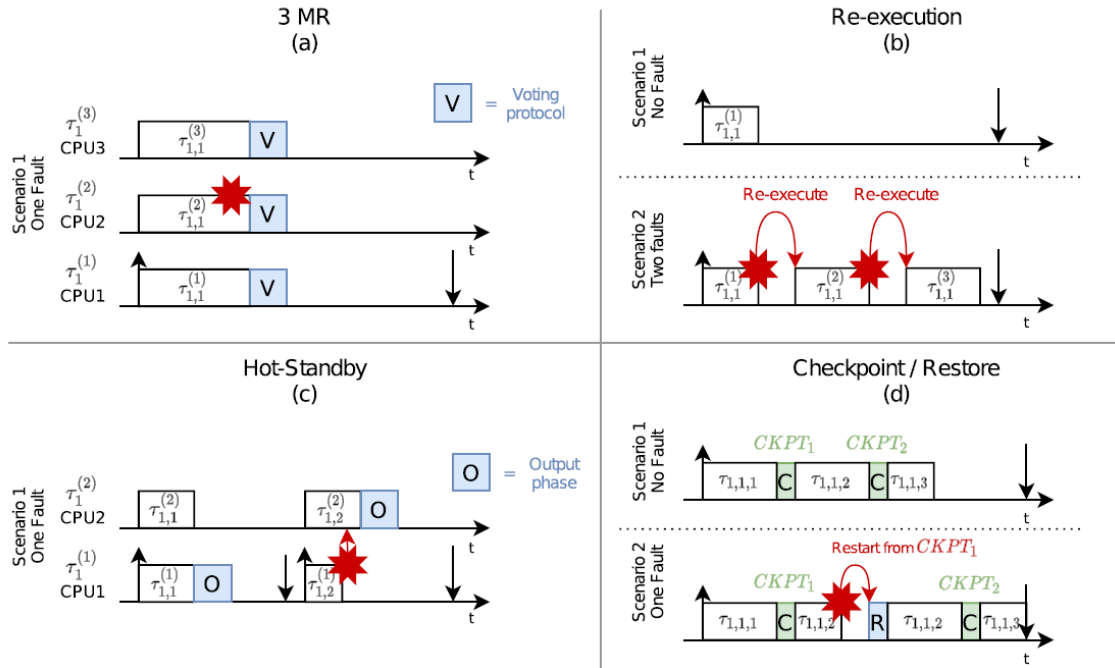


Figure 2.2: The four main fault recovery approaches.

”In (a), (b), and (c), the symbol $\tau_{i,j}^{(k)}$ represents the j^{th} job of the i^{th} task, and k identifies the redundancy or re-execution job. Instead, in (d), $\tau_{i,j,k}$ is the k^{th} part of the job $\tau_{i,j}$. In the depicted example, the job $\tau_{1,1}$ is composed of three parts” - Figure 1 of [4]

2.3.3 Fault Detection

Even though it is not the main topic, neglecting fault detection would be just as bad as neglecting any part of the domain. To highlight this, we can note that *NMR* does recover from an error occurring but it can *not* detect the underlying fault. Preventing the faults as soon as possible save the whole recovery process and thus saving the additional computational power, possibly missed deadlines (in mixed critical systems for example) or even heat-induced wear out.

Hardware

- **Fail-signal processor** is used in multi-processor architectures. A processor complete the task and another one checks for abnormalities (time or result) on the first one.
- **Error detecting code** (EDC): Detects errors (for example checksum). It is used in the memory, the bus and on networks.
- **Watchdogs**: Set a watchdog for the timing constraint.

-

TODO 2: ERROR CORRECTING CODE?

Software

Hardware has the benefit of not affecting execution time because it works in true parallel. It can run checks or backups at the same time as the main task without slowing things down. For software, we have to consider the worst-case execution time of tasks along with the extra time used by the checks added to each job. Software can use multithreading to help, but the extra cost is still there.

2.3.4 Fault Injection

Designing and implementing a system without tests cases is not imaginable. To this end, fault injection is being used. The idea is quite simple, put the system under various stress tests to check its tolerance to faults. This allows the use of a critical micro processor with no more reluctance than necessary. These injections are made using four different techniques. [9, 4]

- **Hardware**-based: Connect a device to mess with the system, for example with voltage spikes or imputing RF disturbances. A simple approach, yet effective.
- **Software**-based: Use a program to mess with the memory allocation of the system, for example allocating random or predefined segments. It is also also fairly simple.
- **Simulation**-based: The hardware is *simulated* and faults are injected through registers using pseudo-random/predefined patterns. This gets more complicated.
- **Emulation**-based: The hardware is fully *emulated* by being written on a Field-Programmable Gate Array (FPGA). This allows host to inject faults in a transparent way (like a real fault would occur).

2.4 System's criticality

Critical systems are nearly everywhere today, from aviation [17] (DAL) to hospitals. A lot of the critical embedded systems are in the Internet of Things (IoT). To address such variety, we can use the Mixed-Critical models. These models consist on prioritizing the tasks with higher importance. For example, a task that needs to check if the distance with the car in front of you is large enough is more important than the one that checks the temperature. If the former fails, a crash might happen as if the latter fails, temperature might go up by 0.1°C. The formal way of doing this is to map a number (the priority) to tasks, 0 being the most critical ones and as the number increases, the priority lowers.

The most common, to my knowledge, mixed critical model is the *Dual* [18]. It consists of two priority :

- HI-critical
- LO-critical

As the names suggest, the HI-criticality stands for tasks that need to be finished in time (with a high priority) and LO-critical is used to describe tasks that can be dropped or can miss a deadline (low priority). The software failure model of a task is usually considered in two senses:

- the logical sense: the task produces an incorrect output
- in the timing sense: the task does not produce a correct output by the end of its deadline

However, all the industries (aviation, automobile, medicals, railway, spaces, industrials, etc.) have their own priority scale. All of them are, to some extent, a specification of a simple Mixed critical system with task priorities. For example, aviation with *DO-178B* (or C) with the DAL priority that goes from A to E, A being the most critical and E being the least.

2.5 Cybersecurity

Cybersecurity is also a main point of attention in this field (as it is a field itself). This is even more valid with the growth of the IoT in general on this past decades and this growth increased with the new arrival of Artificial Intelligence (AI) [8].

TODO 3: THREAT MODELS [19]

.

2.5.1 Trusted Execution Environments (TEE)

A Trusted Execution Environment (TEE) [20] is a secure part of the processor that runs code and handles data in isolation from the rest of the system. The goal is to protect sensitive operations from malware, faulty software, or even privileged system components like the OS or hypervisor. TEEs are often used for secure key storage, encryption, authentication, and digital rights management. It contains enclaves and is often run on *ARM TrustZone* for mobile devices [10].

2.5.2 Enclave

Enclaves provide isolated execution environments that protect sensitive code and data even if the Operating System (OS) is compromised [21]. They rely on hardware-level support (TEE)

to keep the enclave memory hidden from other software, reducing the attack surface. Enclaves help contain faults, data and prevent them from spreading beyond their boundary, adding both security and resilience. It is used on mobile for example to provide a safe environment for banking, cryptographic or even biometric data storage. This is to avoid any third party software being able to access confidential data.

2.5.3 Code obfuscation & diversification

Code obfuscation and diversification make it harder for attackers to understand or predict the system's behavior. Obfuscation transforms the code into a version that works the same but is harder to analyze (static analysis). Diversification changes the memory layout, instruction set, or data locations across runs or devices. These techniques slow down attacks and can stop malware that depends on known patterns, making the system more fault-tolerant against targeted exploits (Zero-day vulnerability for example).

2.5.4 Rewinding & Domains

A current approach to address attacks is to first detect the attack and then roll-back the system to a previous (supposedly) safe state by replications or checkpointing. As much as this mechanism works to ensure reliability of the system, the *resilience* and *availability* when it is under an attack is not the focus. This last point enables an attacker to use the delay of roll-backing the system as an attack vector. To solve this, a software library has been created for 64-bits x86 processors : *Secure Domain Rewind and Discard (SDRaD)* [22]. The underlying logic consists of using *Domains*, compartmentalized and secured spaces and using a domain for each critical task. Thus, when an error occurs, the corrupted domain can be *discarded* (terminated) and the error can be handled. In this scenario, the targets services are Commercial Off-The-Shelf (COTS) processors [22, 5, 4] (ie. processors that don't need hardware specifications or specific changes in order to work) for web services with multiple clients. The plurality of the clients is where the danger lies as if one client is malicious, it can affect the others. That is where the SDRaD mechanism comes in place, each client or subroutines of the clients could be stored in different domains making a great isolation in-between clients. The program using the library can rewind the states of the domains discarded to avoid propagation and this makes the possible Denial of Service (DoS) attack outdated as the states are saved frequently and the whole system does not need to restart from scratch (using caches services for example).

2.5.5 Data & Control flow

Protecting data flow and control flow is key to stopping many types of (dynamic) attacks. Data flow checks make sure sensitive data does not leak or get overwritten. Control flow is the instructions order of execution for the CPU. Hardware or software guards can stop faults caused by memory corruption, buffer overflows, or instruction hijacking. Together, these methods help prevent faults from becoming full system breaches.

2.5.6 Obfuscation measurement

A derivation of this is the layout obfuscation, it consists only on destructuring the program. This is mainly done to prevent a human reading the code base and reverse engineer the whole system (thus gaining informations and understanding of possible breaches). This is a one way approach as when the structure is deleted (when classes are merged for example or comments and name deleted) it cannot be found afterwards. *Potency* and *Resiliency* are used to measure this [11], the

former is to measure human readability and the latter is used to measure how well the system resists to automated decompilers/deobfuscators/deassemblers [9].

2.5.7 Limitations

One of the goals is to make the system more resilient to attacks and thus to rise the cost of developing attacks that could harm the system [9] (this is seen as control flow). Another one is to prevent someone with malicious intent to comprehend the system (static analysis) or even if they do, to prevent them understand the flow of the program as it runs (dynamic analysis). However, as for every domain (in computer science), there are limitations. These inherent limitations are that no system is full proof.

2.5.8 Background work

TODO 4: RELATED WORK THAT'S ALREADY DONE, THAT I'M BASING MYSELF OF.

[22, 10]

Chapter 3

Problem Statement and Objectives

Now that everything is, roughly speaking, well defined. Although the whole domain is very interesting and always needs specific studies on various topics, it needs to be narrowed down in order to suit a Master thesis.

First cut is the type of *redundancy*. For the purpose of the research, the tests should be done using several, not yet decided, *timed* redundancy methods. This allows to measure the viability and the relevance of the work done.

Second cut is the type of systems that needs to be modeled. My focus will be *mixed critical* system, as they are great at modeling real life prioritization.

Third cut is the type of faults. As diving in any type of fault could be a thesis in itself, the faults will be reviewed only for what they are and not necessarily for their cause. Thus, *all three types* will be looked upon.

Next cut is the channel that will be use. In other terms, *hardware* and *software*. As they are both important and linked in various ways. The focus will not only, at least initially, be on one of them but rather on *both* and the implication that each have on the other. For example, Enclaves that need restricted access and create a waterfall of complexity in the design and implementation has on impact on both hardware and software.

Fourth cut is about the *cybersecurity* perspective. As the industry around this domain is mostly critical (aviation, automobile, space or even medical), there is a high risk with someone getting access to something they shouldn't have. Considering that, we must ensure that the faults occurring, possibly from a malicious source, does *not break the system nor give informations* about it. Not much from a static analysis (reverse engineer) perspective but rather from a *control flow* one. To make it more precise, on TEEs when a fault occurs, the system can try to change of enclave as if it was a *Domain*. Then the system can be rewinded and set to a safe state while not being unavailable.

Fifth choice is the platform. There exists a lot of different possibilities when it comes to implementing (and testing) a system. The chosen platform to implement it is Keystone ([12, 23, 24]), it is an open framework used for architecturing TEEs on a RISC-V architecture.

The final choice is to try an implement the whole SDRaD library in a TEE, making an equivalence between domains and enclaves. As the enclaves are subregions of a TEE, they are isolated and secure as the definition of a Domain using Private Keys for Userspace (PKU) and Memory Protection Keys (MPK) (with register mechanism and much more [22]).

In conclusion, the research should focus on *Fault recovery by rewinding enclaves in a TEE on a RISC-V architecture (Keystone)* and extending existing works.

TODO 5: ADD GOALS

Chapter 4

Keystone Structure

”Keystone is an open framework for architecting trusted execution environments (TEEs) based on hardware enclaves. Keystone aims to provide an easy way of architecting and building a customized TEE for an arbitrary RISC-V platform with a simple workflow. This allows the TEE to be highly optimized for better performance, smaller trusted computing base (TCB), and programmability with a given workload and a threat model.” - 2.2. Keystone Basics [24]

Its implementation can be found on the corresponding github repository: [Keystone repository](#) [12] . The corresponding documentation can be found at : <https://docs.keystone-enclave.org/> [23, 24]

4.1 RISC-V

RISC-V is a free and open-source (in contrast to x86 and ARM) Instruction Set Architecture (ISA). Due to its reduced instruction set, it has become a popular architecture for embedded systems and microcontrollers as it allows to reduce power consumption, code size, and memory use. [25]

RISC-V has three privilege levels (2.1.2. RISC-V background [24, 26]):

- U-mode: User processes
- S-mode: Kernel
- M-mode: Bootloader-firmware

M-mode is the highest as it controls physical resources and interrupts. It is analogous to microcode in Complex Instruction Set Computer (CISC) ISAs (x86). However, unlike microcode, it is implemented in existing programming languages (C) and toolchains (gcc). This makes it much more accessible.

The running software can thus not be interfered or interrupted by lower modes. In Keystone, the M-mode is used for:

- the Security Monitor (SM)
- the Trusted Computing Base (TCB)

As all of these are software implementation. It is much easier to implement, debug and fix than it would be on hardware.

4.1.1 Physical Memory Protection (PMP)

”Physical Memory Protection (PMP) is a part of the RISC-V Privileged Architecture Specification which describes the interface for a standard RISC-V memory protection unit.” - [27] It allows the M-Mode to control physical memory access. PMP is ruled by a set of Control Status Registers (CSR) to restrict physical memory access for U and S mode and is configured by the M-mode.

PMP as a variable number of entries depending on the platform it is on. The one in our interest is QEMU, its virtual machine have 16 entries, each entry is defined by one or more CSRs. It works on physical addresses, the configuration of the subsequent virtual addresses can be let to the S-mode without compromising security. Even with different hardware implementation, the standard guarantees basic functionality.

More importantly, PMP is used by the SM to create memory isolation and can be dynamically configured by the M-mode (using CSRs).

4.1.2 Security Monitor (SM)

Is the root trusted software (and thus runs in M-mode). It is used to form the TCB and provides :

- Memory Isolation (PMP)
- Remote attestation
- Enclave thread management
- PMP synchronization
- Side-Channel Defense (FU540 module)

Being a software program, it must be verified first by hardware. This is done by the *Root of Trust* (RoT). At boot time, it first measure the SM, generates a keypair for remote attestation and then signs the public key. After that, the boots is resumed.

The OS and the enclaves can call SM functions using the Supervisor Binary Interface (SBI). Each enclave use a region (section) of memory, each of these regions are defined by PMP entries.

The SM uses by itself, two PMP entries (SM space, untrusted memory) each of the active enclaves consum one entry.

4.1.3 Interrupts and Exceptions

Any interruption or exception is received by the M-mode (by default). It thus have the full authority over the CPU scheduler. Some of its control can be delegated to S-mode. Using a lower level to catch exception such as page fault allows them to be handled quicker as the M-mode has a lot of workload. S-mode is great to handle page faults and system calls because they are frequent.

U and S modes access memory via virtual addresses while M-mode operates exclusively on physical addresses. In fact, the OS cannot tamper with the page table of the enclaves, this is because of PMP restrictions on the OS.

4.2 Basics

4.2.1 Overview

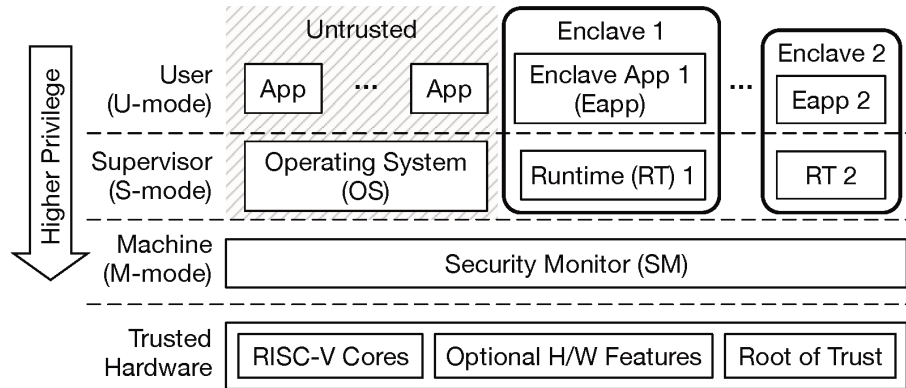


Figure 4.1: Keystone overview (2.2.1 [24])

The keystone environment is made of several, unequal and concurrent, components. The diagram shown in figure 4.1 highlights a privilege hierarchy, going from most to least privileged:

- **Trusted Hardware:** That is composed of *supposedly* trustworthy and contains RISC-V standards cores compatible with Keystone and a Root of Trust (for the SM machine). This part can also bring optional features working with memory (cache partitioning, encryption, cryptographic randomness, etc.). For these features to be usable, the SM requires platform specific plug-ins (FU540 for example).
- **M-mode:** The SM, with a small TCB is the communication between the OS and the Enclaves.
- **S-mode:** Composed mainly of the OS and then each RunTime (RT) part of enclaves are all separated to preserve isolation. The RT part of the enclaves manage all the system call, interruptions, memory management and traps. the
- **U-mode:** Is composed of all the untrusted apps (supervised by the OS) of the computer and all the Eapp part of the enclaves. Note that the enclaves, even in this mode, are encapsulated

4.2.2 Workflow

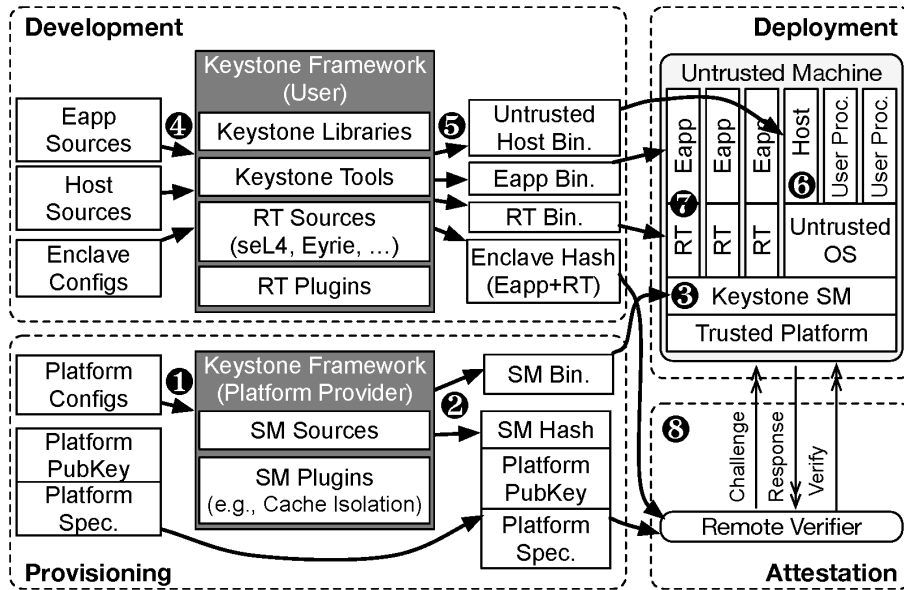


Figure 4.2: Keystone workflow (2.2.2 [23])

Keystone is a two-way framework. It is customizable by both the developers (of the enclaves) and by the platform producers. This means that building on it works in two different ways. The former is the one that is in our interest. Even more, the need to develop a host "supervisor" of the enclave and interface to keep track inbetween runs will add a step.

The basic workflow is to build the eapps, the host and then *possibly modified* runtime binaries. Then, they can (but don't have to), be packaged together into a file (4,5). The files are then deployed (6,7) into the remote system.

Keystone also allows the enclaves to do a remote attestation (2.2.2.3. [24]).

4.3 Enclaves

4.3.1 Enclave lifecycle

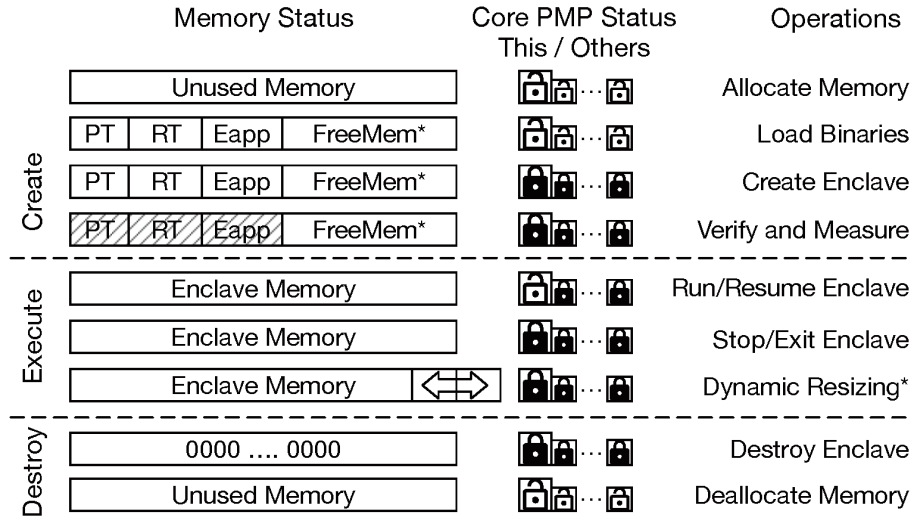


Figure 4.3: Enclave overall lifecycle (2.2.3 [24])

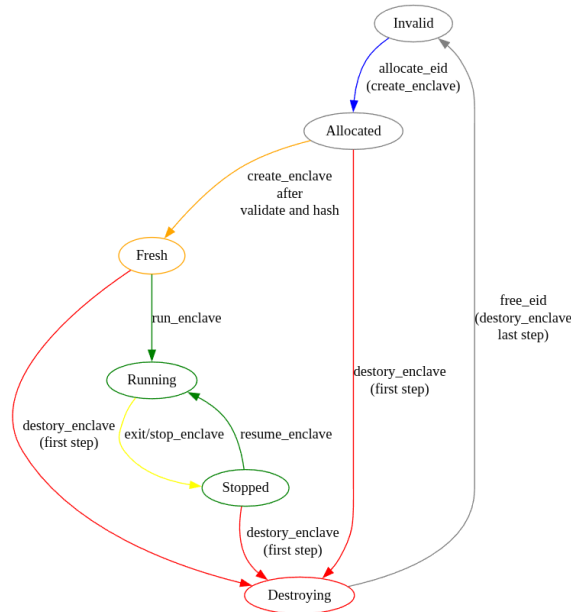


Figure 4.4: Current Enclave Lifecycle (1.9. Keystone Security Monitor[24])

There are three main states for the enclave to be in ((2.2.3) [24]):

- **Creation:** The (untrusted) host calls the SM with the Enclave Private Memory (EPM) allocated and the enclave's Page Table (PT). The SM then receives the host's call and encapsulate the EPM using a PMP entry. The entry is then shared among the cores so that the EPM is sealed from other cores. The SM then checks, right before execution, the state of the enclave (in case of an error).
- **Execution:** Still on the host side, it makes a call to the SM. This time to execute the enclave and passing it to on of the SM's core. The SM thus relieves a PMP entry and the execution

starts. The enclave can be exited and resumed at any time in the execution. Each time the PMP permission is switched to maintain isolation.

- **Destruction:** As well as exiting and resuming the enclave, the host can destroy it. For that, the host calls the SM. the later clears the EPM and then relieve the PMP entry. Last step is for the host to reclaim the newly available memory.

4.4 Quick EMUlator (QEMU)

QEMU is a free and open-source emulator and vizualizer [28]. It has three different operating modes:

- The user emulation that runs a single program.
- The system emulation which emulates a (and possibly several) full system, including preipherals and different architectures like RISC-V. This is the mode used for Keystone.
- The hypervisor support to act as a virtual-machine manager (VMM) or a backend emulation-device for virtual machines.

4.4.1 Eyrie runtime

Eyrie is Keystone’s reference enclave runtime, it runs inside the enclave in trusted mode, manages memory and basic system-calls. It sits between the enclave Eapp and the SM interface [29, 26]. The module is not a submodule of Keystone, this allows modularity and force manual updates [24].

4.4.2 FireSim

”FireSim is an FPGA-based cycle-accurate simulator for RISC-V processors. Using FireSim, you can test Keystone on open-source processors” (1.2.2 [24]). It is an alternative run (for Keystone’s usage) to QEMU as it simulates the hardware [30].

4.5 Edge calls

An Edge call is a function that enters or exits the system. A visual representation is that it is at the *edge* of it, meaning that the method goes from inside to outside and the other way around.

In Keystone, it is used to communicate between the host (unsafe) and the enclave (safe). This is accomplished using shared memory between the two. More precisely using Outbound CALLs (OCALL). However, this is done by using offsets in memory region, using solely offsets (neglecting virtual address pointers) [24, 26].

4.5.1 OCALL lifecycle

To make things clear, we’ll use an example of usage. If we want to print something from inside the enclave, we need to :

- First, decide, with the host (at implemtation time), for the edge call corresponding to print: `OCALL_PRINT_VALUE`.
- Next, stock the value input `val` (*enclave* $\rightarrow$ *host*) we want to print.
- Optional, we decide of a return buffer value `ret` (*host* $\rightarrow$ *enclave*).

- Now make the OCALL:

```
ocall(OCALL_PRINT_VALUE, &val, sizeof(val),
      &ret, sizeof(ret))
```

Now the runtime part allocates an `edge_call_t` structure in shared memory. Then fills the call type and copies the value in the another part of the shared memory using an offset. The last part for the enclave now is for the runtime to exit the enclave without destroying it. This is done with an `SBI_CALL` expressing an ocall.

Now the kernel resumes, checks for the enclave and sees a pending ocall. The execution is thus passed to the host. The host read the `edge_call_t` (destroying it at the same time) and notice that it is `OCALL_PRINT_VALUE`. The host then uses the same offset to read the buffered value and calls `_print_value` with the previously read value as an argument. After that, as for the enclave call, the `ret` buffer can be filled (if it was not 0 in the original ocall) with with data in the shared memory. The `edge_call_t` status is set to `SUCCESS` and the edge call is done for the host.

An `SBI_CALL` is emitted and the runtime re-enters the enclave. The runtime checks for a returned value (if the return buffer wa not 0 at the start). If so, it then uses the offset to put the data in `ret` and then resumes the wrapper code and passes any return value to the first ocall function.

Note that this can also be used to provide networking operations [26].

Since the implementation is assymetric, the host will have to map the edge calls that it could receive to wrapper functions and be able to handle them while running the main loop. The enclave with just send the OCALLs and wait for a possible response from the host.

Host-side

The edge call wrappers are working in parallel to the main loop. These wrappers are implemented as such:

```
// 1. Linking the edge calls to the functions
// this is done in configure_enclave()

// binds ocalls to the enclave
enclave.registerOcallDispatch(incoming_call_dispatch);
register_call(OCALL_PRINT_BUFFER,
              print_buffer_dispatch); // input
register_call(OCALL_SAVE_CHECKPOINT_BLOB,
              save_checkpoint_blob_dispatch); // input
register_call(OCALL_LOAD_CHECKPOINT_BLOB,
              load_checkpoint_blob_dispatch); // output

// 2. Basic edge call structure
void edge_call_wrapper(void *buffer)
{
```

```

// 3. Checks the call sanity
if (edge_call_args_ptr(edge_call, &arg_ptr, &arg_size) != 0
    || arg_size == 0)
{
    edge_call->return_data.call_status = CALL_STATUS_ERROR;
    return;
}

/* 4. Wrapper logic */

// 5. Setup return data
ret_val = /* some logic */
uintptr_t data_section = edge_call_data_ptr();
memcpy((void*)data_section,
        &ret_val,
        sizeof(type_to_return));

if (edge_call_setup_ret(edge_call,
                        (void*)data_section,
                        sizeof(type_to_return)))
{
    call_status = CALL_STATUS_BAD_PTR;
}

// 6. Exiting
edge_call->return_data.call_status = call_status;
return;
}

```

Enclave-side

In contrast to the host, the edge calls in the enclave are sequential and in the main loop.

```

// The edge functions are :
// output
int save_checkpoint(uintptr_t stack_anchor, size_t anchor_len);
unsigned long ocall_print_buffer(char* data, size_t data_len);

// input
int load_checkpoint(struct rewind_checkpoint *checkpoint);

// 1. Basic edge call wrapper structure

```



```

auto edge_call_wrapper(int argc, char** argv)
{
    // 2. OUTPUT functions
    {
        /* output wrapper logic */
    }

    // 3. OCALL
    auto retval;
    ocall(OCALL_ENUM_NUMBER,
          data,
          data_len,
          &retval,
          sizeof(retval));

    // 4. INPUT functions
    {
        /* input wrapper logic */
    }

    //5. Exit
    return (output ? retval : some_value);
}

```

Note that no standard library is used in the enclave as the [Keystone tutorial \(7.1.1 \[23\]\)](#) explicitly states that `printf` is not a secure I/O interface. Considering this, small functions to format numbers into strings (`char*`) have been implemented.

4.6 Data sealing

Keystone has a sealing protocol. Using this, an enclave can save data in an untrusted, non-volatile, memory outside of its memory range. It works by enabling the SM to compute a deterministic key from the RoT for the enclave to use. The key works both for sealing and unsealing data. The deterministic operation is based on the device, a given enclave and a given SM running on a given processor will always yield the same key. Therefore, a combination including any differing component will give a different key [\[24, 26\]](#).

A simple example of usage is given in the github repository ([Keystone repository \[12\]](#)) at `examples/tests/data-sealing/data-sealing.c`:

```

char *key_identifier = "identifier";
struct sealing_key key_buffer;
int ret = 0;

/* Derive the sealing key */

```

```
ret = get_sealing_key(&key_buffer, sizeof(key_buffer),
                    (void *)key_identifier,
                    strlen(key_identifier));
```

This snippet is implemented in the enclave side. Note that there is an *identifier*, this allows the enclave to ask for several keys. Each of them are unique (and deterministic) for a given identifier.

Another subroutine for an enclave is to ask for an attestation using an SBI call `attest` that can be passed to the host using an edge call. The SM will generate and sign an attestation for the enclave. More informations about this can be found in the documentation at 5. *Attestation* [24].

4.6.1 Key-Hierarchy

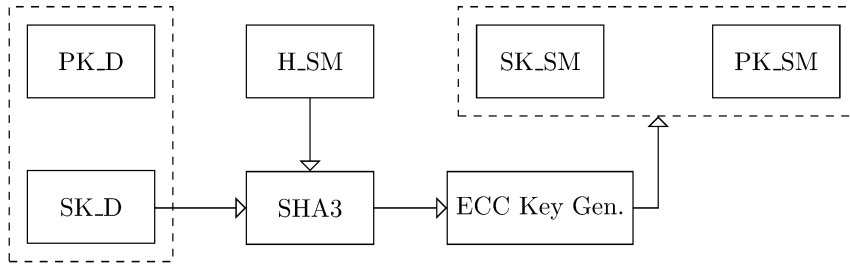


Figure 4.5: Key-Hierarchy (6.1. Keystone Key-Hierarchy [24])

The base of this hierarchy is at the left-most. This is the asymmetric key pair (public and private) of the processor (`PK_D`, `SK_D`). Computing the private key with the SM hash (`H_SM`) which is derived from the RoT in `SHA3`. Thus, as long as the RoT is safe, so is the SM's measurements. The result of the computation is used as a scalar for the Elliptic-Curve Cryptography (ECC) Key Generator. The final result is another key pair. This is the SM key pair (`PK_SM`, `SK_SM`).

4.6.2 Key Derivation

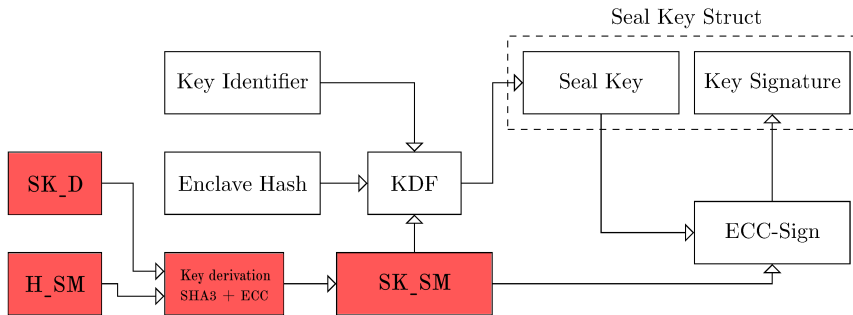


Figure 4.6: Key Derivation Diagram (6.2 Sealing-Key Derivation [24])

The red blocks are what we already computed in the Key Hierarchy 4.6.1 subsection.

Now that the SM has a key pair, it can derive keys for the enclaves. This is done using three components:

- `SK_SM`: SM private key, it ensures that the key is bounded to the SM (and thus the RoT).
- The Enclave Hash, it ensures that the key is bounded to the enclave.
- A *key Identifier* (4.6), it is an additional input for the enclave to be able to derive multiple keys.

The three components are computed in a Key Derivation Function (KDF). The resulting sealing key is then signed using ECC and then placed in the Seal Key Struct (implemented as `sealing_key`):

```
struct sealing_key {
    uint8_t key[SEALING_KEY_LENGTH];
    uint8_t signature[SIGNATURE_SIZE];
};
```

4.7 Compilation

As shown in the documentation, compilation in Keystone is, at first sight, a simple task (1.2.1.3. Compile Sources [24]). However, all of its different, standalone, components (4.4.1, 4.4) make it very modular and thus more complex.

The basic command to compile the source files is:

```
make -j$(nproc)
```

This is divided in 3 different parts:

- `make`: Runs the Makefile.
- `-jN`: "Specifies the number of jobs (commands) to run simultaneously." - Manual page Ubuntu 20.04
- `$(nproc)`: the `$()` is a shell substitution for the `N`, the output of the `nproc` command gives the number of available processing units and is placed in the `$()` substitution.

All together, the Makefile is executed using all the available processing units. This will build all the components and generate a bootable image, the output build in `build-$(PLATFORM)$BITS`.

Also, there are possible options for it, these options are given below (presented with the default parameter) and can be given in the terminal using:

```
OPTION1=VALUE1 OPTION2=VALUE2 (...) make -j$(nproc)
```

4.7.1 Rebuilding Changed Components

We can include specific directories to be cleaned in order to avoid stale sources:

```
BUILDROOT_TARGET=<target>-dirclean
```

The new part is the targeted directory (`<target>`), this could be :

- keystone-examples
- keystone-sm
- keystone-runtime
- all
- etc.

When the compiler notices a stale, a warning message will be shown :

```
>>> (WRN) Stale build directory detected for keystone-examples!
>>> (WRN) You should build keystone-examples-dirclean to remove them.
>>> (WRN) Not doing so may leave your build in an inconsistent state.
```

For this example, the following build command should be ran:

```
BUILDROOT_TARGET=keystone-examples-dirclean make -j$(nproc)
```

4.7.2 Platform

We can configure the platform to build. The default is `generic` and only supports qemu 4.4.

```
KEYSTONE_PLATFORM=generic
```

4.7.3 Bits

We can choose either RV32 or RV64.

```
KEYSTONE_BITS=64
```

4.7.4 Buildroot Config file

Sets the configuration file for the buildroot to use.

```
BUILDROOT_CONFIGFILE=qemu_riscv$(KEYSTONE_BITS)_virt_defconfig
```

Chapter 5

System Model and Design

This chapter describes the concrete implementation, the choices and the assumptions that have been made.

5.1 Preliminary setup and selection

The first step is to have a strong, yet modular, idea of the concept to be implemented. The goal is to obtain domains that can be checkpointed and the data that has been sealed needs to be accessible in another one. The domains discussed here are the ones from the "Rewind & Discard: Improving Software Resilience using Isolated Domains" [22]. We can make an easy parallel between enclaves and domains as they are executed in an isolated execution context with controlled memory (4.1.1) from the outer, untrusted, world. The SM (4.1.2) is an enclave factory and provides them with, as previously mentioned, controlled memory.

Furthermore, several enclaves of the same binary (compiled source file) can be initialized. This is important for the key generation of an enclave (4.6). In the case of a fault occurring in an enclave, the host can see the return code of the enclave and ask for the SM to create another instance of that same enclave (using PMP entries). The two instances will (as the key generation is deterministic [24, 26]) have the same resulting sealing key. This is true if we consider the identifier to be the same for both, as it is implemented in the binary of the enclave's eapp, the statement is supposedly satisfied.

For the checkpointing scheme, the enclave uses the edge calls (4.5) already implemented in keystone. The checkpointed data is sent and retrieved (to and from the host's memory space) using the enclave's key through two custom edge calls. To ensure that the data isn't tampered, there is a layer on top of Keystone's encryption scheme. The layer adds a tag and an Initialisation Vector (IV) for the enclave to check if the unsealed data corresponds to the sealed one. Only the last checkpoint is saved.

TODO 6: CITE PAPERS

Finally, some rewind models can have modified (often degraded) version in case of a discard. This is done to prevent attacks using the computation power and overloading a server by making it fail a task indefinitely for example. In this project, there is only one binary source (the compile code) and it doesn't implement any fault assessment, the enclave behaves as if the fault caused a fatal crash

and stops immediatly, returning an error. However, we address this in the future work section 7.5.4.

5.2 Threat Model

A threat model helps to clarify the hypothesis, the goals and the limitations compared to the real, unsafe, world.

5.2.1 Trusted

- Enclave code, runtime and memory access (Keystone TEE, RISC-V, PMP isolation)
- Enclave-derived sealing keys (via `get_sealing_key`, bound to enclave measurement)

5.2.2 Adversarial

- Host / OS / everything outside the enclave
- The channel carrying the sealed blob (host storage, OCALL buffer, any transport)
- Physical access implied by power loss ([22, 10]), attacker can trigger crashes/power-cuts

5.2.3 Attacker Capabilities

- Can read/write/replay/corrupt any sealed blob stored on the host
- Can trigger enclave stops (or even destroy) when incoming edge calls
- Can crash power/execution at arbitrary points

5.2.4 Security Objectives

1. **Confidentiality**: checkpoint contents unreadable to host
2. **Integrity/authenticity**: tampered blob is detected, rejected and leaks no information (response time analysis)
3. **Atomicity**: no undefined state from interrupted checkpoint writes
4. **Unclonability**: can't clone device from checkpoint data alone

5.2.5 Accepted Limitations

- Replaying and rollback is out of scope. This is because the host is the only link to the outside world and is not to be trusted.
- Edge call manipulation such as enclave stall and destruction not in the scope for the same reason.

5.2.6 Assumptions

The attacker may obtain arbitrary access to process (host) memory and may trigger run-time attacks that corrupt application state, but cannot modify code pages of the enclave under a $W \oplus X$ policy [22, 19]. The attacker is also assumed to detect or exploit software-level vulnerabilities, while transient execution attacks, hardware-level fault injection, and other hardware or firmware-level attacks are out of scope and assumed to be covered by the Keystone security implementation.

In the Keystone setting, we additionally assume the enclave isolation and the underlying secure world mechanisms protect enclave code and secrets against direct software manipulation, and that the host is not trusted with enclave state recovery details.

We further assume that the checkpointing and rewind mechanism operates securely, that is: a state external to the protected domain may be compromised, but the protected recovery boundary must remain trustworthy. For the intermittent-device context, the checkpointed state may be exposed to physical disruption such as power loss or tampering with external storage. The enclave only restores the checkpoint on restart and does not base its execution on the sealed checkpoint it sent unless there is a fault and thus a, possibly corrupted, rewind.

Enclave execution is assumed deterministic given identical entry state (no host-controlled inputs, per Attacker Capabilities). Under this assumption, IV/seq reuse across crash-and-restart is safe: identical plaintext re-encrypted under the same IV leaks nothing. This safety does not hold if the enclave itself introduces non-determinism (uninitialized memory, internal RNG/timer reads, etc.) before the point of divergence from a prior checkpoint_seq.

5.2.7 Requirements

The system must allow the application to continue operation after a detected fault or attack by restoring a trusted state and resuming execution from a safe point. The recovery mechanism must preserve memory integrity after rewind, and isolation must ensure that faults in one compartment do not compromise other protected compartments or the logic responsible for transitions and recovery.

An enclave state must remain protected across enclave transitions, that recovery metadata must be authenticated, and that the host must not be able to forge or tamper with a checkpoint without detection.

5.3 Implementation

So far, this thesis has mainly covered concepts 2, ground work [10, 22, 12] and limitations. All of these are used to produce the implementation of the rewind enclave mechanism.

The files that are implemented and described can be found on the [GitHub repository](#). Also note that the repository setup is described at the end of the `README.md` (at the root of the repository itself). All of these files are also available in the Appendix 8.

Implementing the rewind example is done in two different places, the `eapp/` and `host` sub-folders. The `eapp` is, as shown is the Keystone Basics 4.2, the trusted part and would be considered a *transient domain* [22]. A transient domain can exit (abnormally or not), it doesn't retain data upon exiting and is closed from the outer world (in both writing and reading). In comparison, the `host` would be considered a *persistent domain* as it keeps the enclave data (the sealed checkpoint), it cannot look into an enclave's memory region (except for the shared one given by the SM). However, the host is **not** trusted in itself.

To address this security issue, the checkpoint is sealed (using the sealing key provided by the

SM) and uses a tag. This ensure that the host can neither read it or tamper with it. Upon unsealing a the checkpoint, the enclave checks its tag so see if it fits the data. If not, the data is discarded and the enclave restarts the computation from the beginning.

The discard and rewind is done by asking the SM to destroy the enclave, free the PMP entry and then allocate a new entry on the the same binary to create the rewinded version of the enclave. This new enclave's instance will ask the host (using and edge call) for any saved, sealed, checkpoint. If the host has one it will copy the blob in the shared memory region and the enclave will consume it to try and restore the last checkpoint. If not, the enclave will start from scratch. Note that the first enclave also does this protocol (as the same code is being used).

Host-side

There is only one execution changing macro, other setups are the edge calls :

- `MAX_RUNS`: Decides how many tries the host should do before stopping and considering the job failed.
- `OCALL_PRINT_BUFFER = 1`: Links the OCALL 1 to the print (input) wrapper.
- `OCALL_LOAD_CHECKPOINT_BLOB = 8`: Links the OCALL 8 to the load checkpoint (output) wrapper.
- `OCALL_SAVE_CHECKPOINT_BLOB = 9`: Links the OCALL 9 to the save checkpoint (input) wrapper.

The rewinding protocol in the `host.cpp/.hpp` file 8 looks like:

```
// 1. setup :  
// - the macros in the CMakeList  
// - the edge calls number to be mapped  
  
// 2. Setup the enclave parameters (memory-wise)  
Params params;  
uintptr_t ret = 1;  
auto retry_counter = 0;  
  
// 3. Beginning the execution loop  
while (ret != 0 && retry_counter < MAX_RUNS)  
{  
  
    // 3a. Configure the enclave  
    // enclave.init() && linking edge calls to  
    configure_enclave(enclave, params, argv);  
  
    //SM destroy the enclave upon completion  
    enclave.run(&ret);  
}
```



```

}

// 4. Exiting
return ret;

```

The basic edge call wrapper structure is explained in the edge call section of the keystone architecture 4.5.1. The actual implementation of each wrapper is detailed in 5.3.1.

Enclave-side (EAPP)

As for the host, there is only execution changing macro, other setups are the (same) edge calls. Note that the edge calls must be the same for the host and enclave in order to efficiently communicate.

- EAPP_RUNS: Decides how many iterations the enclave should do before stopping and considering the job done.
- OCALL_PRINT_BUFFER = 1 output
- OCALL_SAVE_CHECKPOINT_BLOB = 9 output
- OCALL_LOAD_CHECKPOINT_BLOB = 8 input

The rewinding protocol in the `rewind.c` file 8 looks like:

```

// 1. Setup :
// - the macros in the CMakeList
// - the edge calls number to be mapped

// 2. Load the structs
struct rewind_state state = {0, 1, 0}; // a, b, counter
struct rewind_checkpoint checkpoint;
struct fault_model f_m = get_default_model();

// 2a. check for an existing checkpoint save
if (load_checkpoint(&checkpoint) == 0)
{
    // 2b. try and restore the saved checkpoint if there is one
    restore_checkpoint(&state, &checkpoint)
}

// 3. Beginning the execution loop
for (; state.counter < EAPP_RUNS;)
{

    // Fault happens before the end of the computation
    // to mimic stall in the process.
    if (fault_should_trigger(&f_m))

```

```

{
    // More explanations in Fault section
    EAPP_RETURN(16);
}

// Fibonacci
auto next = state.a + state.b;
state.a = state.b;
state.b = next;
state.counter++;

save_checkpoint((uintptr_t)&state, sizeof(state));
}

// 4. Exiting
EAPP_RETURN(0);

```

5.3.1 Checkpoint

In order to rewind or discard an enclave, there must be a checkpointed state that is saved and loaded on demand (by the enclave, as defined by Keystone 4). as for the overall implementation, the two sides have their own wrapper logic as they have an asymmetric relationship. The host supervises the enclave and the latter is, once running, sealed from the former 8. However, as soon as the enclave submit an edge call, the host could (assuming malicious intentions) destroy the enclave or stall it indefinitely. This is bounded by Keystone structure and is, as noted in the threat model 5.2, not in the scope of this thesis.

One could implement a direct call from the enclave to the SM to external safe memory with, for example, table pages protected by outside PMP entry or another "data enclave" that has a direct communication channel with the "runtime" one. The problem in our scenario is that the host is the only link to the outside world and is, by definition, also not to be trusted. This limitation being addressed, the saved checkpoint must be sealed and must be checked in order to not introduce unsafe data into the enclave's memory.

Host-side

The print buffer will not be detailed as it is already implemented in the Keystone basic examples and is thus already explained in the documentation [23]. Furthermore, the host keeps the last saved checkpoint blob in the `inline vector<uint8_t> saved_blob` variable. This will be used for both loading and saving.

Notice that this variable, is one of the possible attack vectors for a malicious program that would corrupt the host (in both reading and writing). The edge calls are, by definition and because they are linked to a specific function, another attack vector.

1. load wrapper

```
void load_checkpoint_blob_dispatch(void* buffer)
{
    // 1a. gather OCLL parameters
    struct edge_call* ecall = (struct edge_call*)buffer;

    // 1b. checks if there is a saved checkpoint
    if (saved_blob.empty())
    {
        ecall->return_data.call_status = CALL_STATUS_ERROR;
        return;
    }

    // 1c. looks into shared memory
    void* ret_buffer = (void*)edge_call_data_ptr();

    // 1d. save it
    memcpy(ret_buffer, saved_blob.data(), saved_blob.size());

    /* 1e. if TESTING enabled -> tamper */
    if (HOST_TESTING)
    {
        tamper_checkpoint_blob(saved_blob);
    }

    // 1f. return and exit with ok status
    edge_call_setup_ret(ecall, ret_buffer, saved_blob.size());
    ecall->return_data.call_status = CALL_STATUS_OK;
}
```

2. save wrapper

```
void save_checkpoint_blob_dispatch(void* buffer)
{
    // 2a. gather OCLL parameters
    struct edge_call* ecall = (struct edge_call*)buffer;
    uintptr_t arg_ptr;
    size_t arg_size;

    // 2b. check parameters sanity
    if (edge_call_args_ptr(ecall, &arg_ptr, &arg_size) != 0 ||
```

```

        arg_size == 0)
    {
        ecall->return_data.call_status = CALL_STATUS_ERROR;
        return;
    }

    // 2c. save (sealed) checkpoint
    saved_blob.resize(arg_size);
    memcpy(saved_blob.data(), (void*)arg_ptr, arg_size);

    // 2d. exit
    ecall->return_data.call_status = CALL_STATUS_OK;
}

```

Enclave-side

As for the host, the enclave must keep an anchor in order to be able to save/load a checkpoint. This is extern struct rewind_state *state_anchor.

1. load wrapper

```

int load_checkpoint()
{
    // 1a. sends an OCALL to the SM
    if (ocall(OCALL_LOAD_CHECKPOINT_BLOB,
             NULL, // data to send
             0, // size of this NULL data
             &checkpoint_blob, // data to retrieve
             sizeof(checkpoint_blob)) != 0)
    {
        eapp_print("No saved checkpoint");
        return -1;
    }

    // 1b. try to unseal the retrieved data
    if (open_checkpoint_blob(&checkpoint_storage,
                           &checkpoint_blob) != 0)
    {
        return -1;
    }

    // 1c. increment sequence and exit
    /*
    the sequence is incremented to mimic what would happend
    if the the enclave was at this iteration and just saved

```

```

    (the increment is done after the save).
    */
    checkpoint_storage.checkpoint_seq++;
    return 0;
}

// Note: The enclave must then use
// restore_checkpoint()
// to restore the decrypted checkpoint into the state_anchor

```

2. save wrapper

```

int save_checkpoint()
{
    // 2a. setup parameters
    uintptr_t stack_sp;
    uintptr_t stack_anchor;
    uintptr_t snapshot_sp;
    uintptr_t snapshot_end;
    size_t snapshot_len;
    struct checkpoint to_save; // temporary built checkpoint

    // 2b. load and check the parameters
    if (state_anchor == NULL)
    {
        eapp_print("Invalid checkpoint anchor");
        return -1;
    }

    memset(&to_save, 0, sizeof(to_save));
    stack_sp = read_stack_pointer();
    stack_anchor = (uintptr_t)state_anchor;
    snapshot_end = stack_anchor + sizeof(*state_anchor);

    snapshot_sp = snapshot_end > STACK_SNAPSHOT_SIZE
        ? snapshot_end - STACK_SNAPSHOT_SIZE
        : stack_sp;

    // keep the snapshot aligned to the current stack
    if (snapshot_sp < stack_sp) {snapshot_sp = stack_sp;}

    if (snapshot_end < snapshot_sp)
    {
        eapp_print("Invalid snapshot anchor");
    }
}

```

```

        return -1;
    }

    snapshot_len = snapshot_end - snapshot_sp;

    if (snapshot_len > STACK_SNAPSHOT_SIZE)
    {
        eapp_print("Stack snapshot too large");
        return -1;
    }

    // 2c. save the current data into the tmp variable
    to_save.checkpoint_seq = checkpoint_storage.checkpoint_seq;
    memset(to_save.stack_data, 0, sizeof(to_save.stack_data));
    memcpy(to_save.stack_data+(STACK_SNAPSHOT_SIZE-snapshot_len),
           (void *)snapshot_sp,
           snapshot_len);

    memset(&checkpoint_blob, 0, sizeof(checkpoint_blob));

    // 2d. seal blob
    if (seal_checkpoint_blob(&checkpoint_blob, &to_save) != 0)
    {
        return -1;
    }

    // 2e. output OCALL
    if (ocall(OCALL_SAVE_CHECKPOINT_BLOB,
             &checkpoint_blob, // data to save
             sizeof(checkpoint_blob),
             NULL, 0) != 0) // no data to wait from the host
    {
        eapp_print("failed to save checkpoint");
        return -1;
    }

    // 2f. increment checkpoint_seq and exit
    checkpoint_storage.checkpoint_seq++;
    return 0;
}

```

5.3.2 Data encryption

To preserve the security objectives, the enclave implements a cryptographic protocol Appendix 8.

The authentication and encryption keys are both derived using the function `derive_checkpoint_material()` using `Keystone get_sealing_key()` 4.6. This function is cached after the first usage as the keys will never be changed afterwards. The workflow can be described as such:

1. Save

- a) `derive_checkpoint_material()` asks the SM sealing API for key material bound to scheme label (`identifiers[]`).
- b) The returned material is two independent AES-256 keys:
 - `enc_key` for confidentiality (CTR stream encryption)
 - `auth_key` for integrity/authenticity (CBC-MAC)
- c) `encrypt_stack(payload)` AES-CTR-encrypts the plaintext payload and the iv.
- d) `compute_tag()` MACs the ciphertext checkpoint payload.
- e) The payload and tag are copied into the host-facing blob.

```
// 1a-b . Derive checkpoint key material
if (derive_checkpoint_material() != 0) { return -1; }

// 1c. encrypt_stack(payload):
// AES-CTR-encrypt payload, build iv from checkpoint_seq
uint8_t iv[AES_BLOCK_SIZE] = {0};
memcpy(iv,
        &checkpoint->checkpoint_seq,
        sizeof(checkpoint->checkpoint_seq));
memcpy(iv + sizeof(checkpoint->checkpoint_seq),
        &checkpoint->checkpoint_seq,
        sizeof(checkpoint->checkpoint_seq));

uint8_t mac_input[AES_BLOCK_SIZE +
                  sizeof(struct checkpoint)];
size_t mac_len = AES_BLOCK_SIZE + payload_len;
memcpy(mac_input, iv, AES_BLOCK_SIZE);
memcpy(mac_input + AES_BLOCK_SIZE, payload,
        payload_len);

aes_key_setup(enc_key, enc_schedule, AES_KEY_BITS);
aes_encrypt_ctr(mac_input, mac_len,
```

```

        mac_input,
        enc_schedule,
        AES_KEY_BITS,
        iv);

// 1d. compute_tag(): AES-256-CBC-MAC over the ciphertext and iv
uint8_t zero_iv[AES_BLOCK_SIZE] = {0};
aes_key_setup(auth_key, auth_schedule, AES_KEY_BITS);
aes_encrypt_cbc_mac(payload, payload_len,
                    tag,
                    auth_schedule, AES_KEY_BITS,
                    zero_iv);

// 1e. Copy ciphertext payload, iv and tag into the blob
memcpy(blob->iv, iv, sizeof(iv));
memcpy(blob->sealed, payload, sizeof(payload));
memcpy(blob->sealed + sizeof(payload), tag, sizeof(tag));

// the blob is then sent to the host using the
// previously described OCALL_SAVE_CHECKPOINT_BLOB edge call

```

2. Load

- a) Derive the same enc_key/auth_key again from the same context.
- b) Split the opaque blob into ciphertext, iv, and tag.
- c) Recompute the expected tag over the ciphertext and given iv.
- d) Compare expected tag with the stored tag; if mismatch, reject.
- e) Decrypt the ciphertext.
- f) Rebuild the plain checkpoint.

```

// 2a. Derive the same enc_key/auth_key
// again from the same context
if (derive_checkpoint_material() != 0) { return -1; }

// 2b. Split the opaque blob into ciphertext, iv, and tag
memcpy(iv, blob->iv, sizeof(iv));
memcpy(ciphertext, blob->sealed, sizeof(ciphertext));
memcpy(tag, blob->sealed + sizeof(ciphertext), sizeof(tag));

// 2c. Recompute the expected tag over the ciphertext
compute_tag(ciphertext, sizeof(ciphertext), expected_tag);

```



```

// 2d. Compare expected tag with the stored tag;
// reject on mismatch
if (memcmp(expected_tag, tag,
           sizeof(expected_tag)) != 0) {
    eapp_print("checkpoint authentication failed");
    return -1;
}

// 2e. Decrypt the ciphertext,
// same iv derived from checkpoint_seq
aes_key_setup(enc_key, enc_schedule, AES_KEY_BITS);
aes_decrypt_ctr(ciphertext, sizeof(ciphertext),
               payload,
               enc_schedule, AES_KEY_BITS,
               iv);

// 2f. Copy the plain decrypted checkpoint
memcpy(checkpoint, payload, sizeof(*checkpoint));

```

Data structures

Three data structures separate three distinct roles in the checkpoint/rewind pipeline:

TODO 7: ADD SIZES

```

struct checkpoint checkpoint_storage
struct sealed_checkpoint checkpoint_blob
struct rewind_state *state_anchor

```

- `struct checkpoint checkpoint_storage`: the *plaintext* in-enclave representation of the saved state (e.g. register file, stack snapshot, sequence number). This is the object produced right before sealing and the object reconstructed right after unsealing. It never leaves the enclave in this form.
- `struct sealed_checkpoint checkpoint_blob`: the *host-oriented, opaque* container holding the encrypted payload, the IV, and the authentication tag. This is the only representation of the checkpoint that is allowed to cross the enclave boundary and be stored/-transmitted by the untrusted host.
- `struct rewind_state *state_anchor`: the *runtime pointer/metadata* used by the rewind mechanism to locate and validate which checkpoint(s) are eligible for restoration (e.g. current sequence number, pointer to active checkpoint, rewind policy state). It is not itself sealed; it governs *which* `checkpoint_blob` may be opened and *when*.

Cryptographic primitives and concepts.

- **CBC-MAC**: Message Authentication Code built by running block-cipher CBC encryption over the message and keeping only the final block as the tag. Provides integrity/authenticity, *not* confidentiality. Secure only for fixed-length messages unless additional length-binding is used.
- **CTR (Counter mode)**: block-cipher mode that turns AES into a stream cipher by encrypting successive counter values and XOR-ing the result with plaintext/ciphertext. Provides confidentiality only; requires a unique (IV, key) pair per encryption to remain secure.
- **ENC / AUTH**: shorthand for the two independent purpose-bound keys derived from the sealing key material: `enc_key` (used only for CTR confidentiality) and `auth_key` (used only for CBC-MAC integrity). Key separation prevents cross-purpose attacks between encryption and authentication.
- **IV / Nonce / Seq**: Initialization Vector (or *nonce*, "number used once") is the per-encryption input that randomizes/varies the keystream so identical plaintexts don't produce identical ciphertexts. Here it is derived deterministically from `checkpoint_seq`, a monotonically-assigned sequence number identifying each checkpoint instance. Its uniqueness across all encryptions under the same key is the core security requirement of CTR mode.
- **TAG**: the output of the CBC-MAC over the ciphertext. ie. a fixed-size value that the verifier recomputes and compares against the stored value to detect any tampering with the sealed payload.

5.3.3 Fault Modeling

Advanced fault detection and correction is not in the scope of this thesis. Thus, a simple fault model is implemented : Appendix 8. it is based of of the the SplitMix [31, 32] algorithm. It is a small, object-oriented, non-cryptographic, Pseudo-Random Number Generator (PRNG) and is based on Linear Congruential Generators (LCG) followed by bit-mixing (XOR-shifts) operations. It produces good enough results while being lightweight.

We maintain a 64-bit (SplitMix64) `fault_model` structure with three fields:

- **seed**: initial value for the generator
- **step**: current iteration count
- **period**: inverse of fault probability

The seed and the period can be chosen by modifying the according macros in the `CMakeList 8`. Furthermore, the seed can be either random (based on the runtime cycle counter) or fixed.

- **SEED**: The seed to be picked, useful for testing.
- **FAULT_RANDOMIZE_SEED**: If set to true (by default) then the seed is "randomly" picked, otherwise it is set to `SEED`.
- **PERIOD**: The number of call to the function, on average, before it should return true.

The normal usage is :

```
// 1. setup the macros in the CMakeList

// 2. initialize the fault_model
struct fault_model f_m = get_default_model();

// 3. at each step, trigger a possible fault
// chances are 1/PERIOD
if (fault_should_trigger(&f_m))
{
    // 4. Exit then enclave with abnormal code
    // Keystone::Error::EnclaveInterrupted = 16, the enum
    EAPP_RETURN(16);
}
```

Note that we treat a fault as an error. That is because the goal is to checkpoint and not to catch (or to directly correct) SEUs, bursts or even permanently damaged (worn out) hardware. Malicious data tampering in the enclave is not discussed as Keystone supposedly guarantees data isolation using PMP 4.1.1, the SM 4.1.2 (and thus a RoT). Moreover, other possible fault modeling approaches are discussed in 7.5.3.

5.3.4 Scripts

Building and running scripts are great to make the development quicker and less prone to error. There are three scripts at the root of the project that can be used in order to compile, to run and to do both sequentially:

- `make-examples.sh`: Serves as the compilation (of the examples 4.7).
- `run-rewind.exp`: The most complex one as qemu 4.4 is an emulation, inputs and waiting times (interactions) are expected from the user. Thus, a `.exp` (expect) script is perfectly suited.
- `make-run.sh`: A simple concatenation of the other two.

The **logs** produced can be found in the `build-$PLATFORM$BITS/build.log` file.

Chapter 6

Experimental Setup

The Rewind enclave is evaluated on its overhead computation cycles and data cost. Moreover, the cyber security assumptions are being tested individually.

6.1 Macros

The tests are being conducted using different macros that are all defined in the `CMakeList.txt` following this structure:

```
## Definition, the "some_value" can be left to ""
set(EXAMPLE_MACRO "some_value" CACHE \
    STRING "explanation of the macro")

## Link for the eapp to access it
if(NOT "${EXAMPLE_MACRO}" STREQUAL "")
    target_compile_definitions(${eapp_bin} \
        PRIVATE MACRO_TO_BIND=${EXAMPLE_MACRO})
endif()

## Link for the host to access it
if(NOT "${EXAMPLE_MACRO}" STREQUAL "")
    target_compile_definitions(${host_bin} \
        PRIVATE MACRO_TO_BIND=${EXAMPLE_MACRO})
endif()
```

In case of them not being defined or if the field is left as "" in the `CMakeList.txt`, a fail safe is there in each relevant header file using this structure:

```
#ifndef EXAMPLE_MACRO
#define EXAMPLE_MACRO some_value
#endif
```

There are various ones, three main categories emerge:

- Logging, to enable/disable output from host/enclave.
- Execution, to control the workflow (the number of computation iterations and tries).
- Testing, to enable/disable tests from the host/enclave.

6.1.1 Logging

These macros are not to be confused with testing macros as they are independant from them. One could run the tests of the side with the logging of the enclave or all the tests with no logging (which wouldn't be very interesting).

- `HOST_LOGGING`: enable host printing
- `EAPP_LOGGING`: enable eapp printing
- `EAPP_BREAK_EVEN_CSV_OUTPUT`: output a csv format in order to produce graphs (7.2)

6.1.2 Execution

- `EAPP_RUNS`: sets the number of iteration the enclave does
- `HOST_MAX_RUNS`: sets the maximum number of try before stopping
- `FAULT_PERIOD`: sets the inverse of the chances to trigger a fault per call

6.1.3 Testing

Because the two parts are opaque to each others, tests must be implemented on both sides. These macros enables (or disables) the tests in a branching way.

Host-side

- `HOST_TESTING`: launches the host in testing mode, reduce outputs and run `ANALYSIS_RUNS` iterations to compute averages
- `ANALYSIS_RUNS`: sets the number of runs for the analysis (host side)
- `TEST_TAMPER_MODE`: set the mode of tampering test to do (none/bitflip/truncate/last-byte)

Enclave-side

- `EAPP_TESTING`: launches the execution in testing mode, starts the test sequence
- `FAULT_RANDOMIZE_SEED`: enable/disable random seed for the fault model
- `FAULT_SEED`: sets the seed to start from for the fault model (if `FAULT_RANDOMIZE_SEED=0`)
- `EAPP_AVG_FAULT_TESTING`: enables the fault model average period testing
- `EAPP_BLOB_SIZE_TESTING`: enables the blob size benchmark
- `EAPP_ROUND_TRIP_TESTING`: enables the round trip test
- `EAPP_CYCLE_BREAKDOWN_TESTING`: enables the cycle cost breakdown benchmark
- `EAPP_BREAK_EVEN_TESTING`: enables the break-even threshold benchmark

Manipulating the macros can get confusing as they are independent. One could launch a full set of host testing with `HOST_TESTING=1` and `ANALYSIS_RUNS=100` (to analyze the average runtimes for example) while enabling the `EAPP_TESTING` making the enclave only executing tests (and even more, returning failed tests that the host will try to accomplish). Or trying tests like `TEST_TAMPER_MODE=bitflip` and not setting the `HOST_TESTING` to true. In order to better understand what is going on, it is recommended to enable both loggins macros.

6.2 Test cases

This section defines the test cases used to evaluate the checkpoint and rewind mechanism and explains their relevance. There are two parts: the *Analysis* (6.2.2), where we quantify execution costs and overhead, and the *Cyber security* (6.2.1), where we validate the threat model assumptions and security objectives from 5.2.

6.2.1 Cyber security

Each test focuses on:

- the attacker goal
- the security objective under evaluation (confidentiality, integrity, atomicity, unclonability)
- the behavior of the enclave (e.g., detection and rejection of tampered checkpoints)

6.2.2 Analysis

Each test specifies:

- the enclave workload and macros used setup
- the fault occurrence rate
- the metrics recorded (latency, CPU cycles, memory overhead, final success rate)

Chapter 7

Results and Discussion

In this chapter, we will discuss the computation cost of checkpointing each iterations and the security assumptions and their limitations.

7.1 Static Analysis

The enclave is made to run in discrete time, for our calculations, we will use :

- N the number of runs it is doing before the job is considered done
- K the number of occuring fault during the enclave runs
- $errors$ a set of size K of the indexes of the occuring faults
- C_{event} the total number of cycles used by the *event* (save, load, compute, etc.)

Having these variables states we can compute the theoretical cost of checkpointing compared to restarting the enclave from scratch. A small nuance is that the C_{save} and C_{load} do not include the corresponding edge calls because of their assymetry.

$$C_{checkpoint} = N \cdot (C_{save} + C_{compute}) + K \cdot C_{load} \quad (7.1)$$

$$C_{no_checkpoint} = (C_{compute}) \cdot (N + \sum_{i=0}^K (errors[i]))$$

Note that the sum of the error indexes ($\sum_{i=0}^K (errors[i])$) as an expected value of $K \cdot \lfloor \frac{N}{2} \rfloor$.

This is because the probability of a fault occuring at index i over $[0, N]$ (X_i , which is equal to $errors[i]$) is $\frac{1}{N}$ as X_i is iid. Thus:

$$\mathbb{E}[X] = \frac{N}{2} \quad (7.2)$$

The average (expected value) can be derived using basic probability:

$$\begin{aligned}
\mathbb{E} \left[\sum_{i=0}^K \text{errors}[i] \right] &= \mathbb{E} \left[\sum_{i=0}^K X_i \right] \\
&= \sum_{i=0}^K \mathbb{E}[X_i] \\
&= \sum_{i=0}^K \mathbb{E}[X] \quad (\text{since } X_i \text{ are i.i.d.}) \\
&= K \cdot \mathbb{E}[X] \\
&= K \cdot \frac{N}{2}
\end{aligned}$$

One can test the validity of this statement in the implementation by running the average fault testing and the break-even threshold.

Using that result, $C_{no_checkpoint}$ can be written as :

$$C_{no_checkpoint} = C_{compute} \cdot \left(N + \frac{K \cdot N}{2} \right) \quad (7.3)$$

7.1.1 Computation threshold

Having the equations 7.1 and 7.3, we can compute the threshold. This threshold is obtained when the ratio shown below is **greater or equal to one**.

However, the equations below are taking the *expected* values (average), a single iteration of an enclave of this will **not** follow this exact rule. This is because of the randomness, a simple (yet more than rare) example would be that an iteration with 10 faults have all of them at the first step. The number of runs for $C_{no_checkpoint}$ is then $N + K$ which is not only very much below the average but also equal to $C_{checkpoint}$. The latter being strictly more expensive in cycles will perform more poorly. The expected behaviour for a given K are discussion in Section 7.1.

$$ratio = \frac{C_{no_checkpoint}}{C_{checkpoint}} \quad (7.4)$$

$$ratio = \frac{C_{compute} \cdot \left(N + \frac{K \cdot N}{2} \right)}{N \cdot (C_{save} + C_{compute}) + K \cdot C_{load}} \quad (7.5)$$

$$\begin{aligned}
\frac{C_{compute} \cdot \left(N + \frac{K \cdot N}{2}\right)}{N \cdot (C_{save} + C_{compute}) + K \cdot C_{load}} &\geq 1 \\
C_{compute} \cdot \left(N + \frac{K \cdot N}{2}\right) &\geq N \cdot C_{save} + N \cdot C_{compute} + K \cdot C_{load} \\
C_{compute} \cdot \left(N + \frac{K \cdot N}{2} - N\right) &\geq N \cdot C_{save} + K \cdot C_{load} \\
C_{compute} \cdot \frac{K \cdot N}{2} &\geq N \cdot C_{save} + K \cdot C_{load} \\
C_{compute} &\geq \frac{N \cdot C_{save} + K \cdot C_{load}}{\frac{K \cdot N}{2}}
\end{aligned}$$

The final equation is thus:

$$C_{compute} \geq \frac{2 C_{save}}{K} + \frac{2 C_{load}}{N} \quad (7.6)$$

This confirms that what is seen in the graphs of Section 7.2 :

- For $k = 0$, the model can only approach, from below, the threshold ratio.
- As the C_{save} and C_{load} are rising, the $C_{compute}$ must also go up, this translates to a plateau after the first threshold overpass.
- The number of fault K , that is supposedly much smaller than the number of runs N , which is great (≥ 10.000) as per the goal of long running tasks, makes the C_{load} weight little compared to C_{save}

Given the actual costs of the save and load cycles as shown in 7.2.2. We can isolate the computation needed in order to reach the threshold value.

Assuming that we use:

- $N = 1.000.000$
- $K = 1$ the minimal number to be able to reach a threshold
- $C_{save} = 15.000.000$ as the runtime test is $[15M - 20M]$
- $C_{load} = 15.000.000$ to better isolate the compute cost, we assume that load and save are symmetric

$$\begin{aligned}
C_{compute} &\geq \frac{2 \cdot 15.000.000}{1} + \frac{2 \cdot 15.000.000}{1.000.000} \\
C_{compute} &\geq 30.000.000 + 30
\end{aligned}$$

This result is confirmed by the runtime analysis graphs (and underlying benchmarks) in Section 7.2.1.

7.2 Runtime Analysis

Now that the theory is established, we will focus on the runtime benchmarks and their resulting graphs. In order to avoid outliers and CPU inconsistency, all the tests have been made 1000 times, their results have been divided by the same amount. The following subparts are constituted of three graphs, these graphs are all representing the same output data but the cost of the save and load (C_{save} , C_{load}). Our current implementation save and load costs are ranging from 15.000.000 and 20.000.000 (7.2.2), this is the value given to the first two of each graphs. Third one has a set cost of 50.000.000 for both. The choice has been made to set them to equal values as C_{load} holds less weight for our purpose of long running tasks (7.3), avoiding unnecessary confusion.

Moreover, the compute cost measured in our implementation is contained within $[60k - 200k]$. It is a simple addition, variable switch and an increment and doesn't represent a real life specific application. The main application being very theoretic, we compare different values in order to avoid making false assumptions of the implementation of a specific task in the enclave. We will discuss the relevance of these possible values of the three different costs in the Section 7.4.

7.2.1 Computation threshold

These graphs have been made using the `EAPP_BREAK_EVEN_TESTING` macro along with the `EAPP_BREAK_EVEN_CSV_OUTPUT`.

Firstly, For the fixed K (to 1). The Figure 7.1 and is the *low expectation* runtime, the ratio is met at around $30M$ as shown in the example equation. Figure 7.2 is the *high expectation* runtime, the ratio is met at around $40M$. To be convince of this, just replace the parameters of the equation 7.6:

$$\begin{aligned} C_{compute} &\geq \frac{2 \cdot 20.000.000}{1} + \frac{2 \cdot 20.000.000}{1.000.000} \\ C_{compute} &\geq 40.000.000 + 40 \end{aligned}$$

As for the last one, the Figure 7.3 can be verified if we just replace the parameters:

$$\begin{aligned} C_{compute} &\geq \frac{2 \cdot 50.000.000}{1} + \frac{2 \cdot 50.000.000}{1.000.000} \\ C_{compute} &\geq 100.000.000 + 100 \end{aligned}$$

Secondly for the overview of several possible K ,

Fixed K

First, we fix K to 1 as if it is below that, the threshold cannot be reached and if higher, the threshold will simply be met sooner. Making this value the most interesting one to analyze. These k-fixed graphs also highlight the possible gap between the average, the maximum and the minimum recorded over the 1000 test runs. This contrast to the theoretic bound by showing that even if the average is indeed meeting the threshold, an actual run might not be worth the excess computation.

On the other hand, we now know that the model behaves as intended and is, most likely, reliable when conditions are met.

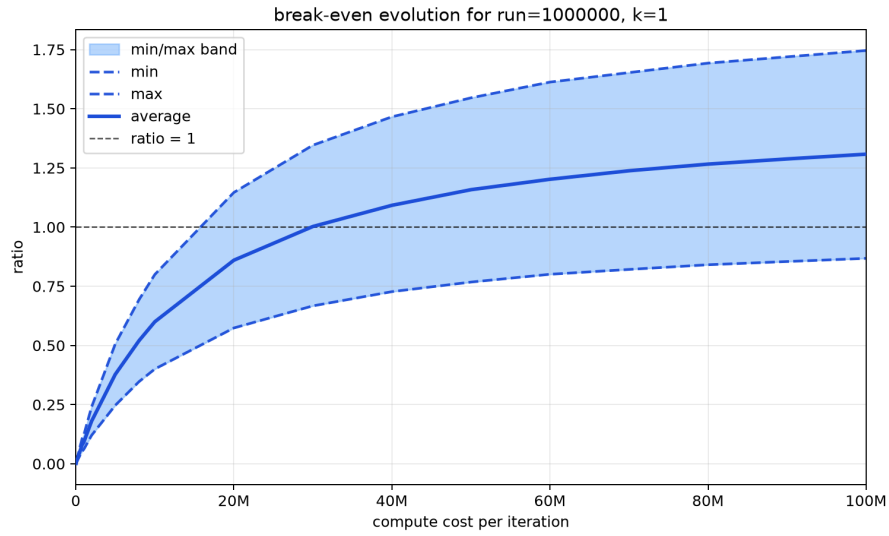


Figure 7.1: Threshold for 1 error over a 1.000.000 runs enclave assuming a 15M cost for load and save

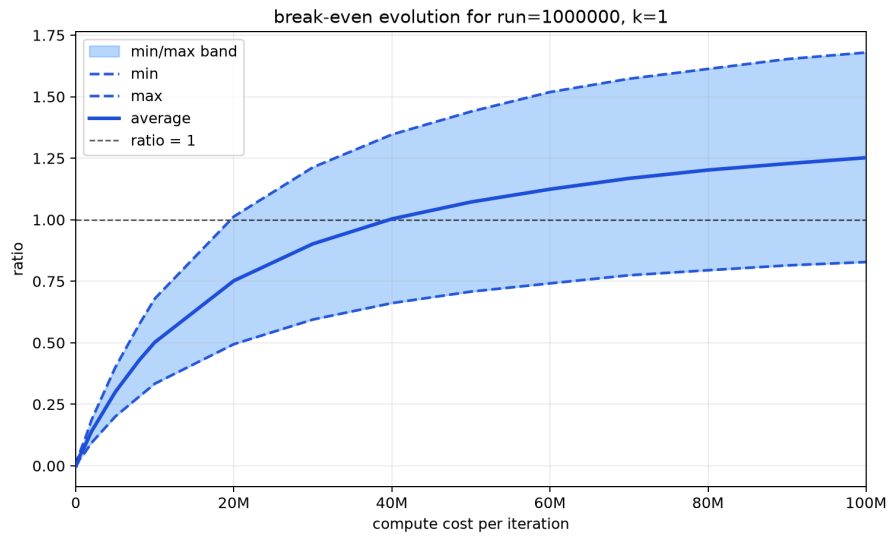


Figure 7.2: Threshold for 1 error over a 1.000.000 runs enclave assuming a 20M cost for load and save

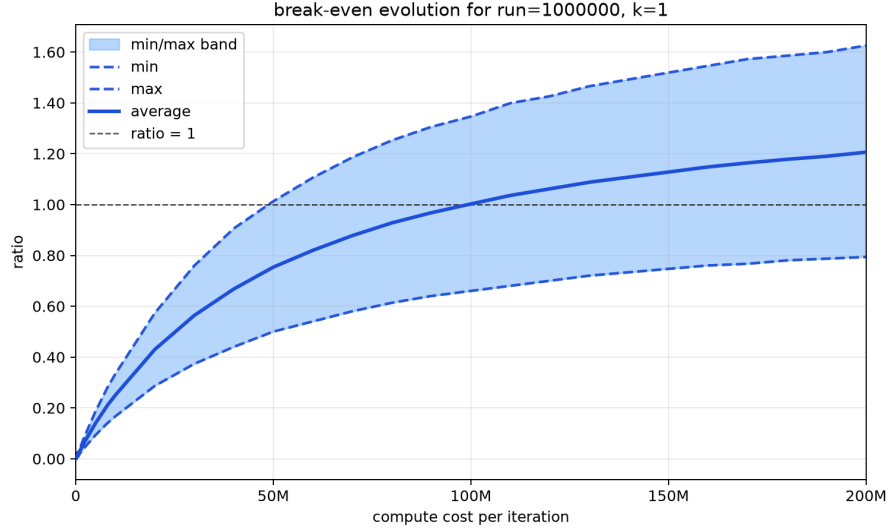


Figure 7.3: Threshold for 1 error over a 1.000.000 runs enclave assuming a 50M cost for load and save

$$\mathbf{K} = [0 - 9]$$

Next we show the result, without the maximal and minimal bounds for the first 9 errors. These ones highlight the plateau effect even when working with possibly more faults. As we explained above, the plateau appears because as we increase $C_{compute}$ to balance the other ones, the two sides equals out and both C_{save} and C_{load} become insignificant. Our intuition can also help us understand that effect as both $C_{checkpoint}$ and $C_{no_checkpoint}$ both use $C_{compute}$. Nevertheless, as K grows, the first slope gets steeper. We can explain that with the effective cost of the save being shrinked by more faults, in both intuition and equation 7.6. The the balance between the two effect gives some kind of saturation slop.

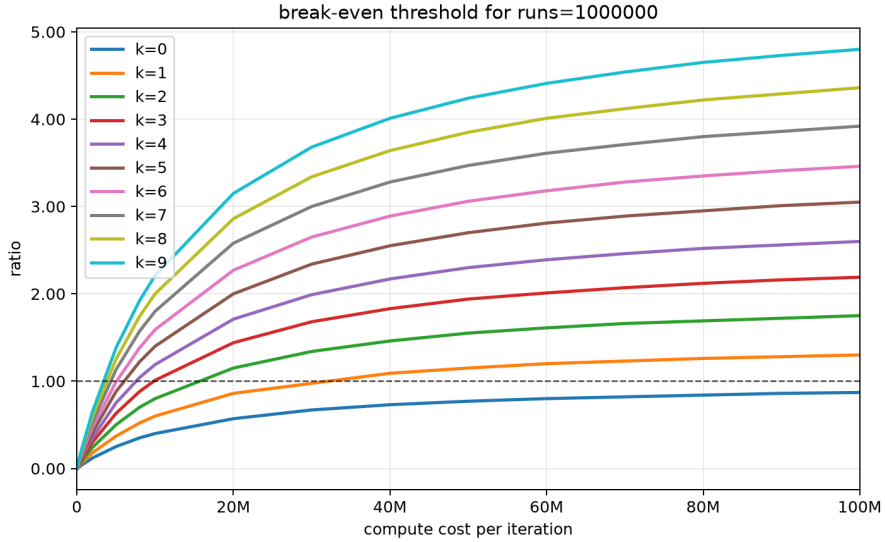


Figure 7.4: Threshold ratio for $k=[0 - 9]$, over a 1.000.000 runs enclave assuming a 15M cost for load and save

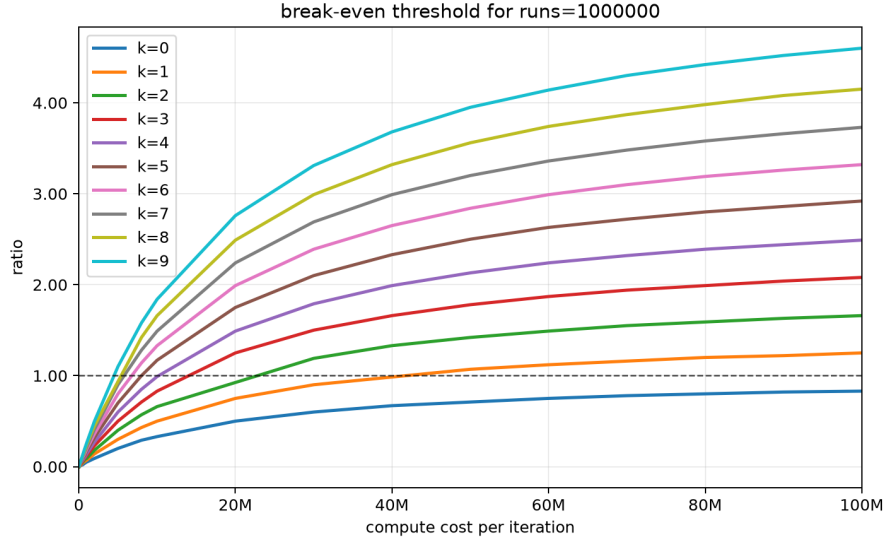


Figure 7.5: Threshold ratio for $k=[0 - 9]$, over a 1.000.000 runs enclave assuming a 20M cost for load and save

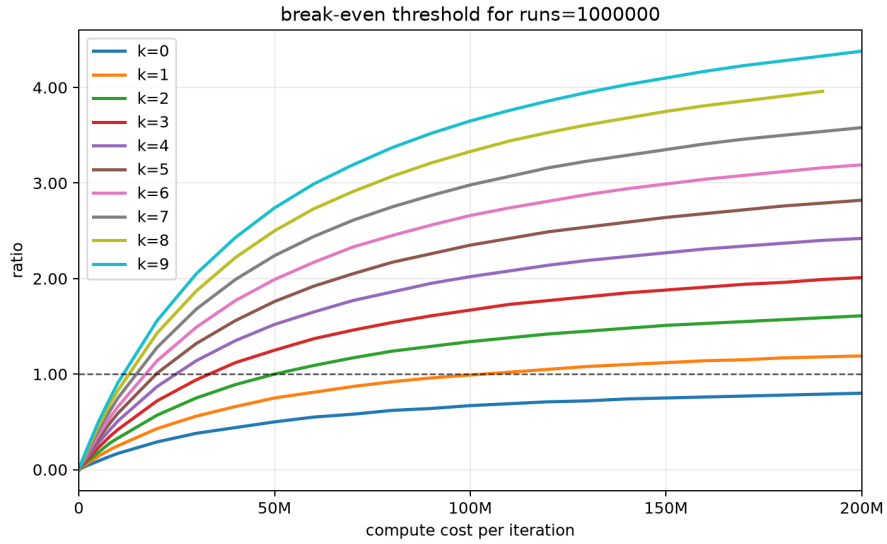


Figure 7.6: Threshold ratio for $k=[0 - 9]$, over a 1.000.000 runs enclave assuming a 50M cost for load and save

7.2.2 Cycle cost per operation

This part is a recap of each possible main operation of the enclave during an iteration. The values below are averaged over 1000 runs. They exclude the cached material derivation from the per-part seal and unseal breakdown.

Table 7.1: Seal cycle cost breakdown

Component	Cycles
Key derivation	21,873
Seal preparation	9,366
AES-CTR encryption	9,698,615
CBC-MAC tag	9,939,972
Blob copy out	16,089
Total seal cost	19,664,042

Table 7.2: Unseal cycle cost breakdown

Component	Cycles
Blob copy in	8,367
CBC-MAC tag recomputation	9,992,689
Tag compare	1,697
AES-CTR decryption	9,702,006
Blob copy out	13,409
Total unseal cost	19,718,168

Table 7.3: Other main operation costs

Operation	Cycles
Computation	275,100

7.3 Cyber security

Now that the computation and data analysis is done, we will address the cyber security assumptions and requirements.

7.3.1 Security objectives

As stated in 5.2, there are four security objectives that will be looked upon. We will discuss of why they hold and what are the limitations. The claims can be checked using the corresponding macros in (Section 6.1).

Confidentiality

The checkpoint is, at any time, opaque from the host. An attacker can, indeed, not read the saved checkpoint because of the `aes_encrypt_ctr()` function provided by Keystone. In our implementation, the rewind state to save is read from the stack, placed into a plain text checkpoint storage and then sealed using the encryption key derived from Keystone data sealing key (Section `get_sealing_key()` 4.6) and the iv (the checkpoint sequence). After that, a tag is done over the ciphertext using the authentication key to block anyone from tampering without notice. The resulting sealed checkpoint blob is then sent to the host along with the iv and the tag.

Integrity

Based on the Confidentiality CBC-MAC and CTR scheme, tampering is handled.

Now we have to consider the possible side-attacks, these are that the attackers tries to tamper with the blob directly, which is not plausible under the scheme *over a fixed length message*. Another attack is to obtain side informations to deduce information. This can be time to compute the tag or reloading the enclave by destroying it at every save edge call. The first one is addressed using a simple, home made compare function that always passes over the whole sequence instead of a memcmp() that stops as soon a one byte is wrong. using memcmp(), or any non constant time comparison function would lead to a time of authentication leak. The second one is, in our case, directly addressed by the inherent deterministic mechanism of the enclave, the initial state, will always be the same as the one being compiled (whatever a and b are). Moreover, the checkpoint sequence also follows this rule. Making a XOR of a possibly same ciphertext with different IVs simply not possible.

Atomicity

The atomicity states that we cannot have an undefind saved state. Based on the two last objectives, a maliciously modified state is not something to scared of. However, could an attacker stops the enclave at the perfect time in order to create a chimera state. The answer is no. In fact, the only real abilities, to this regard, of the attacker is to cut the power of the enclave. The only outcome of this scenario is to also stop the host and the SM, effectively stopping the whole system. The protection of a whole system is not in the scope of this thesis. Nevertheless, the state, checkpoint of this state and sealed checkpoint are being kept separated for this purpose. Whenever one changes, it doesn't always affect the others.

Another idea would be to compromise the PMP hardware itself, this scenario is not in the scope of this thesis as it is assumed to be sound in runtime to malicious programs. Even only destroying the entry would result in a call to the SM, and a move of the enclave from this hardware space.

Unclonability

We consider a device to be cloned if we can replicate the exact behaviour given a sealed state. This is only possible for a program that has the **exact** same hash as the EAPP and is ruled by the SM. Otherwise, the keys won't correspond and the attacker will not be able to read or to use the sealed data. In addition to that, there is no point in time where the plaintext is exposed. We can thus safely rely on Keystone data sealing to make the sealed checkpoint unusable for someone that is not the enclave EAPP.

7.4 Discussion

great, better (less computation) crypto scheme, how many runs, still useful in practice in case of a full stop of several days for examples

TODO 8: TIME REGARDING

TODO 9: THREAT IS CONTAINED BUT HOST IS UNRELIABLE

7.5 Future Work

7.5.1 Mixed criticality

7.5.2 Modern lightweight cryptoscheme

7.5.3 More complex fault model

7.5.4 Enclave Complexity on backup

Chapter 8

Conclusion

TODO 10: CONCLUSION

Bibliography

- [1] Hadi Esmaeilzadeh et al. “Dark Silicon and the End of Multicore Scaling”. In: *IEEE Micro* 32.3 (2012), pp. 122–134. DOI: [10.1109/MM.2012.17](https://doi.org/10.1109/MM.2012.17).
- [2] John von Neumann. “Probabilistic Logics and the Synthesis of Reliable Organisms from Unreliable Components”. In: *Automata Studies*. Ed. by C. E. Shannon and J. McCarthy. Princeton, NJ: Princeton University Press, 1956, pp. 43–98.
- [3] A. Avizienis. “The N-Version Approach to Fault-Tolerant Software”. In: *IEEE Transactions on Software Engineering* SE-11.12 (1985), pp. 1491–1501. DOI: [10.1109/TSE.1985.231893](https://doi.org/10.1109/TSE.1985.231893).
- [4] Federico Reghenzani, Zhishan Guo, and William Fornaciari. “Software Fault Tolerance in Real-Time Systems: Identifying the Future Research Questions”. In: *ACM Comput. Surv.* 55.14s (July 2023). ISSN: 0360-0300. DOI: [10.1145/3589950](https://doi.org/10.1145/3589950). URL: <https://doi.org/10.1145/3589950>.
- [5] Inseok Hwang et al. “A Survey of Fault Detection, Isolation, and Reconfiguration Methods”. In: *IEEE Transactions on Control Systems Technology* 18.3 (2010), pp. 636–653. DOI: [10.1109/TCST.2009.2026285](https://doi.org/10.1109/TCST.2009.2026285).
- [6] Sepideh Safari et al. “A Survey of Fault-Tolerance Techniques for Embedded Systems From the Perspective of Power, Energy, and Thermal Issues”. In: *IEEE Access* PP (Jan. 2022). DOI: [10.1109/ACCESS.2022.3144217](https://doi.org/10.1109/ACCESS.2022.3144217).
- [7] IVÁN SERRANO HERNÁNDEZ. “SOFTWARE-LEVEL REDUNDANCY AND DIVERSITY FOR OBJECT DETECTION IN SAFETY-CRITICAL EMBEDDED SYSTEMS”. MA thesis. Universitat Politècnica de Catalunya (UPC) - BarcelonaTech, 2024. URL: <https://upcommons.upc.edu/entities/publication/0ded2aa4-21b1-4e30-bb35-f5206e1e5010>.
- [8] B. Nagajayanthi. “Decades of Internet of Things Towards Twenty-first Century: A Research-Based Introspective”. In: *Wireless Personal Communications* 123 (2022), pp. 3661–3697. DOI: [10.1007/s11277-021-09308-z](https://doi.org/10.1007/s11277-021-09308-z). URL: <https://doi.org/10.1007/s11277-021-09308-z>.
- [9] Shohreh Hosseinzadeh et al. “Diversification and obfuscation techniques for software security: A systematic literature review”. In: *Information and Software Technology* 104 (2018), pp. 72–93. ISSN: 0950-5849. DOI: [10.1016/j.infsof.2018.07.007](https://doi.org/10.1016/j.infsof.2018.07.007). URL: <https://www.sciencedirect.com/science/article/pii/S0950584918301484>.

- [10] Pietro Chiavassa et al. “Secure Intermittent Computing with ARM TrustZone on the Cortex-M”. In: *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. 2024, pp. 160–168. DOI: [10.1109/EuroSPW61312.2024.00022](https://doi.org/10.1109/EuroSPW61312.2024.00022).
- [11] Antonio Carzaniga, Andrea Mattavelli, and Mauro Pezzè. “Measuring Software Redundancy”. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 1. 2015, pp. 156–166. DOI: [10.1109/ICSE.2015.37](https://doi.org/10.1109/ICSE.2015.37).
- [12] Dayeol Lee et al. “Keystone: An Open Framework for Architecting Trusted Execution Environments”. In: *Proceedings of the Fifteenth European Conference on Computer Systems*. EuroSys’20. 2020.
- [13] John Shalf. “The future of computing beyond Moore’s Law”. In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 378.2166 (2020), p. 20190061. DOI: [10.1098/rsta.2019.0061](https://doi.org/10.1098/rsta.2019.0061). eprint: <https://royalsocietypublishing.org/doi/pdf/10.1098/rsta.2019.0061>. URL: <https://royalsocietypublishing.org/doi/abs/10.1098/rsta.2019.0061>.
- [14] Institute of Electrical and Electronics Engineers (IEEE) is a global community for technologists, helping to shape the systems and standards of tomorrow. URL: <https://www.ieee.org/>.
- [15] Magne Vollan Aarset. “How to Identify a Bathtub Hazard Rate”. In: *IEEE Transactions on Reliability* R-36.1 (1987), pp. 106–108. DOI: [10.1109/TR.1987.5222310](https://doi.org/10.1109/TR.1987.5222310).
- [16] Jens Lienig and Hans Bruemmer. *Fundamentals of Electronic Systems Design*. Vol. 1. Springer International Publishing, Apr. 2017, pp. xiii + 241. ISBN: 978-3-319-55840-0. DOI: [10.1007/978-3-319-55840-0](https://doi.org/10.1007/978-3-319-55840-0). URL: <https://link.springer.com/book/10.1007/978-3-319-55840-0>.
- [17] Andreas Löfwenmark and Simin Nadjm-Tehrani. “Fault and timing analysis in critical multi-core systems: A survey with an avionics perspective”. In: *Journal of Systems Architecture* 87 (2018), pp. 1–11. ISSN: 1383-7621. DOI: <https://doi.org/10.1016/j.sysarc.2018.04.001>. URL: <https://www.sciencedirect.com/science/article/pii/S1383762117304903>.
- [18] Sepideh Safari et al. “On the Scheduling of Energy-Aware Fault-Tolerant Mixed-Criticality Multicore Systems with Service Guarantee Exploration”. In: *IEEE Transactions on Parallel and Distributed Systems* 30.10 (2019), pp. 2338–2354. DOI: [10.1109/TPDS.2019.2907846](https://doi.org/10.1109/TPDS.2019.2907846).
- [19] László Szekeres et al. “SoK: Eternal War in Memory”. In: *2013 IEEE Symposium on Security and Privacy*. 2013, pp. 48–62. DOI: [10.1109/SP.2013.13](https://doi.org/10.1109/SP.2013.13).
- [20] Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. “Trusted Execution Environment: What It is, and What It is Not”. In: *2015 IEEE Trustcom/BigDataSE/ISPA*. Vol. 1. 2015, pp. 57–64. DOI: [10.1109/Trustcom.2015.357](https://doi.org/10.1109/Trustcom.2015.357).
- [21] Gianluca Scopelliti et al. “End-to-End Security for Distributed Event-driven Enclave Applications on Heterogeneous TEEs”. In: *ACM Trans. Priv. Secur.* 26.3 (June 2023). ISSN: 2471-2566. DOI: [10.1145/3592607](https://doi.org/10.1145/3592607). URL: <https://doi.org/10.1145/3592607>.

- [22] Merve Gülmez et al. “Rewind & Discard: Improving Software Resilience using Isolated Domains”. In: *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2023, pp. 402–416. DOI: [10.1109/DSN58367.2023.00046](https://doi.org/10.1109/DSN58367.2023.00046).
- [23] Keystone Enclave Project. *Keystone: Open-Source RISC-V Trusted Execution Environment Framework*. <https://keystone-enclave.org>. Accessed from October 2025 trough August 2026. 2021.
- [24] Keystone Enclave Project. *Keystone Documentation*. <https://docs.keystone-enclave.org/en/latest/index.html>. Accessed 2026-07-26. 2026.
- [25] Andrew Waterman and Krste Asanović. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. Tech. rep. Document Version 20191214-draft. RISC-V Foundation, Dec. 2019.
- [26] Markus Switalla. “A Review of the Keystone Trusted Execution Framework”. In: 2023. URL: <https://api.semanticscholar.org/CorpusID:259367604>.
- [27] SiFive Inc. *SiFive Freedom Metal library*. <https://sifive.github.io/freedom-metal-docs/devguide/pmps.html>. 2019.
- [28] Peter Maydell et al. *QEMU Website Licensing*. Accessed July 2026. 2014. URL: <https://www.qemu.org/license.html>.
- [29] 2022. URL: <https://github.com/keystone-enclave/keystone-runtime>.
- [30] Sagar Karandikar. “FireSim: FPGA-Accelerated Cycle-Exact Scale-Out System Simulation in the Public Cloud”. MA thesis. EECS Department, University of California, Berkeley, Dec. 2018. URL: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2018/EECS-2018-154.html>.
- [31] Guy L. Steele, Doug Lea, and Christine H. Flood. “Fast splittable pseudorandom number generators”. In: *SIGPLAN Not.* 49.10 (Oct. 2014), pp. 453–472. ISSN: 0362-1340. DOI: [10.1145/2714064.2660195](https://doi.org/10.1145/2714064.2660195). URL: <https://doi.org/10.1145/2714064.2660195>.
- [32] Guy L. Steele, Doug Lea, and Christine H. Flood. “Fast splittable pseudorandom number generators”. In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications*. OOPSLA ’14. Portland, Oregon, USA: Association for Computing Machinery, 2014, pp. 453–472. ISBN: 9781450325851. DOI: [10.1145/2660193.2660195](https://doi.org/10.1145/2660193.2660195). URL: <https://doi.org/10.1145/2660193.2660195>.

Appendix

Rewind source code

These files (and their usage) can be found in the repository: [GitHub repository](#) and they are compiled (downloadable) in this pdf.

```
# scripts
make-examples.sh
make-run.sh
run-rewind.sh

# source files
examples/rewind/
  CMakeLists.txt
eapp/
  checkpoint.c
  checkpoint.h
  common.c
  common.h
  crypto.c
  crypto.h
  eapp_test.c
  eapp_test.h
  fault.c
  fault.h
  rewind.c      # enclave file
host/
  host_test.cpp
  host_test.hpp
  host.cpp
  host.hpp
```

eapp/checkpoint

```
examples/rewind/eapp/checkpoint.c
examples/rewind/eapp/checkpoint.h
```

eapp/eapp_test

examples/rewind/eapp/eapp_test.c

examples/rewind/eapp/eapp_test.h

eapp/crypto

examples/rewind/eapp/crypto.c

examples/rewind/eapp/crypto.h

eapp/common

examples/rewind/eapp/common.c

examples/rewind/eapp/common.h

eapp/fault

examples/rewind/eapp/fault.c

examples/rewind/eapp/fault.h

eapp/rewind.c

examples/rewind/eapp/rewind.c

host/host_test

examples/rewind/host/host_test.cpp

examples/rewind/host/host_test.hpp

host/host

examples/rewind/host/host.cpp

examples/rewind/host/host.hpp

CMakeLists.txt

examples/rewind/CMakeLists.txt

Scripts

make-examples.sh

./make-examples.sh

run-rewind.exp

./run-rewind.exp

make-run.sh

`./make-run.sh`

README

README.md